



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

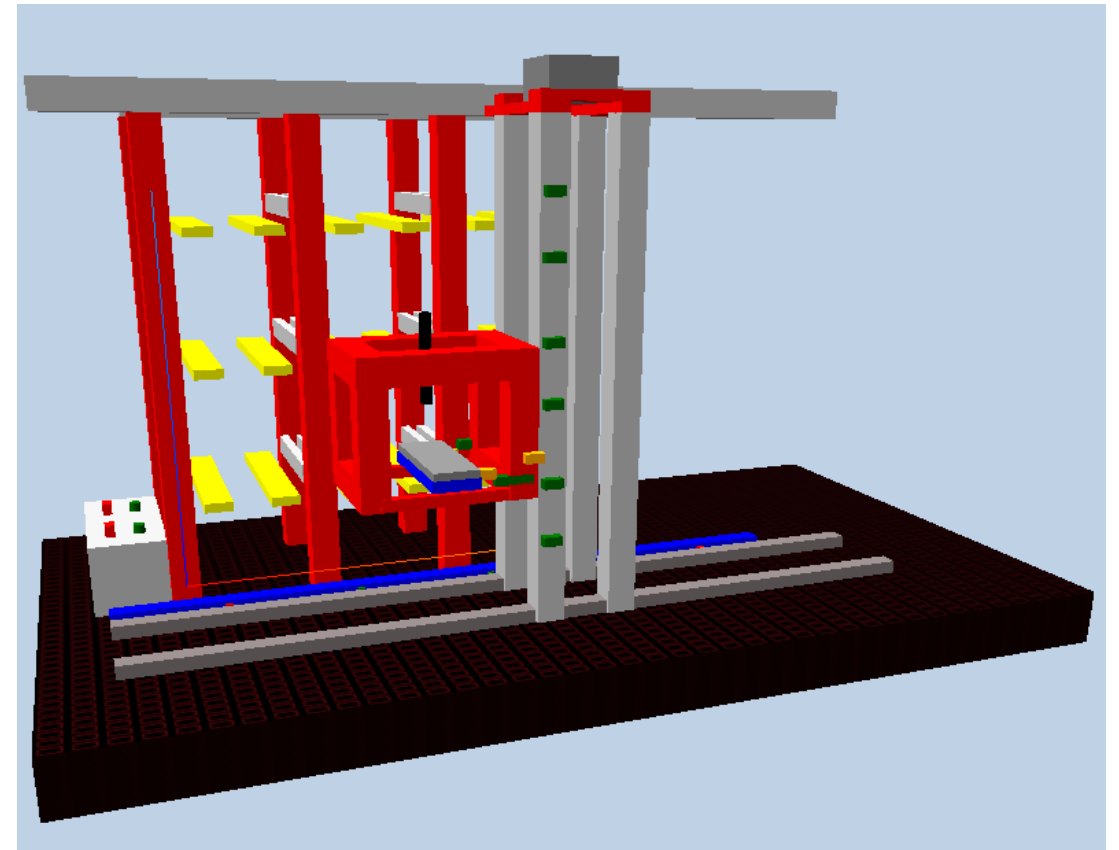
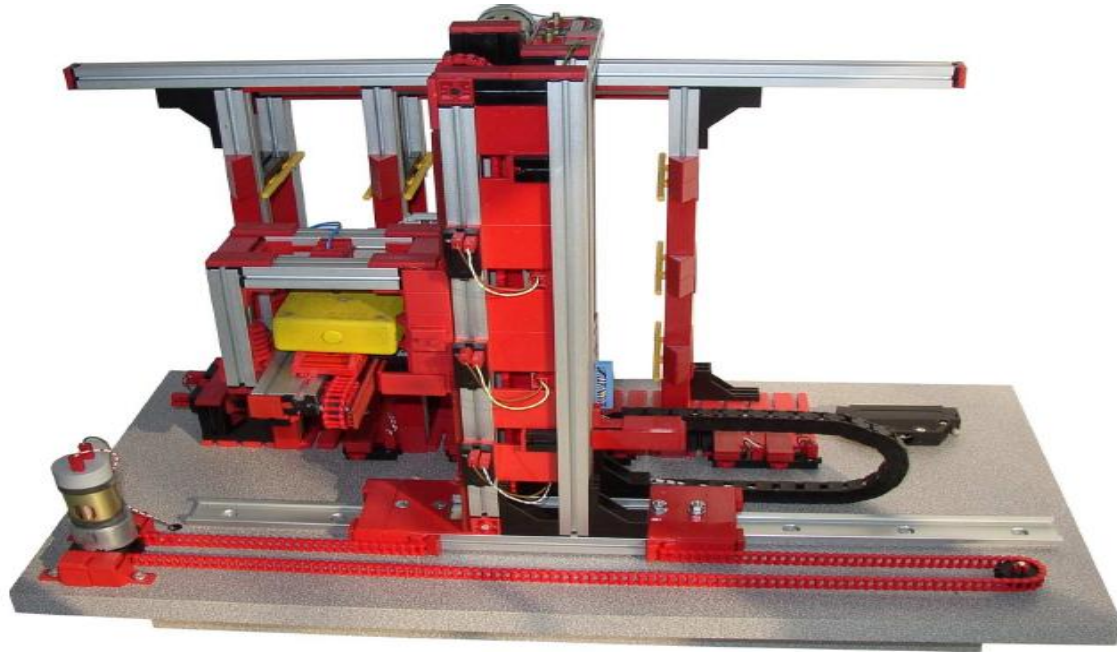
STR

LAB WORK 2 – CLASS 1

**Java SDK and Visual Studio Code Installation
Interaction with Compact Storage Physical
System**

2025 - 2026

LABWORK 2 – COMPACT STORAGE KIT



□ 4 classes

Class 1	SW Installation & Interaction with “physical” system. Small Java tutorial and low-level functionalities implementation.
Class 2	JNI development for interacting with the physical system from Java. Elaboration of both low-level and high-level routines for LabWork2.
Class 3	Introduction to Threads and synchronization and communication mechanisms. Development of the necessary threads for modeling LabWork2 concurrent behavior.
Class 4	Fulfillment of each functional requirement towards the conclusion for LabWork2.



Deadline for Delivery: 24-11-2025

PART 1:

Software installation & configuration

- ❑ Software installation
 - [Java SDK Installation](#)
 - [Visual Studio Code Installation](#)
 - [Small Java tutorial](#)

PART 2:

Interaction with physical system

- ❑ [Introducing the kit “Compact Storage”](#)
- ❑ [Creation of Visual Studio console application project](#)
- ❑ [Low-level functionality](#)
- ❑ [Testing each kit movement](#)

PART 1

Software Installation & Configuration

SOFTWARE INSTALLATION



You will need a computer with the following software (or similar newer versions):

- Java JDK 17

(<https://www.oracle.com/java/technologies/downloads/#java17>)

- Visual Studio Code

(<https://code.visualstudio.com/download>)

Note: If you prefer to use other IDEs such as Netbeans, it will also be acceptable.

The screenshot shows the Oracle Java Downloads page for Java 17. The page header includes the Oracle logo and navigation links for Products, Industries, Resources, Customers, Partners, Developers, Events, and Company. Below the header, there are tabs for Java downloads, Tools and resources, and Java archive. The main content area is titled "Java 19 and Java 17 available now" and contains information about the latest long-term support release for the Java SE platform. It mentions that JDK 19 and JDK 17 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions. It also states that JDK 19 will receive updates under these terms, until March 2023 when it will be superseded by JDK 20, and that JDK 17 will receive updates under these terms, until at least September 2024. Below this information, there are tabs for Java 19 and Java 17. The Java 17 tab is selected, and it shows the "Java SE Development Kit 17.0.5 downloads" section. This section includes a thank you message for downloading the release and a description of the JDK as a development environment for building applications and components using the Java programming language. It also mentions that the JDK includes tools for developing and testing programs written in the Java programming language and running on the Java platform. Below the description, there are tabs for Linux, macOS, and Windows. The Windows tab is selected, and it shows a table of download links for different file formats.

Product/file description	File size	Download
x64 Compressed Archive	171.88 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.zip (sha256)
x64 Installer	152.85 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.exe (sha256)
x64 MSI Installer	151.73 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.msi (sha256)

The screenshot shows the Visual Studio Code download page. The header includes the Visual Studio Code logo and navigation links for Docs, Updates, Blog, API, Extensions, FAQ, and Learn. There is a search bar and a "Download" button. Below the header, there is a message stating "Version 1.72 is now available! Read about the new features and fixes from September." and a close button.

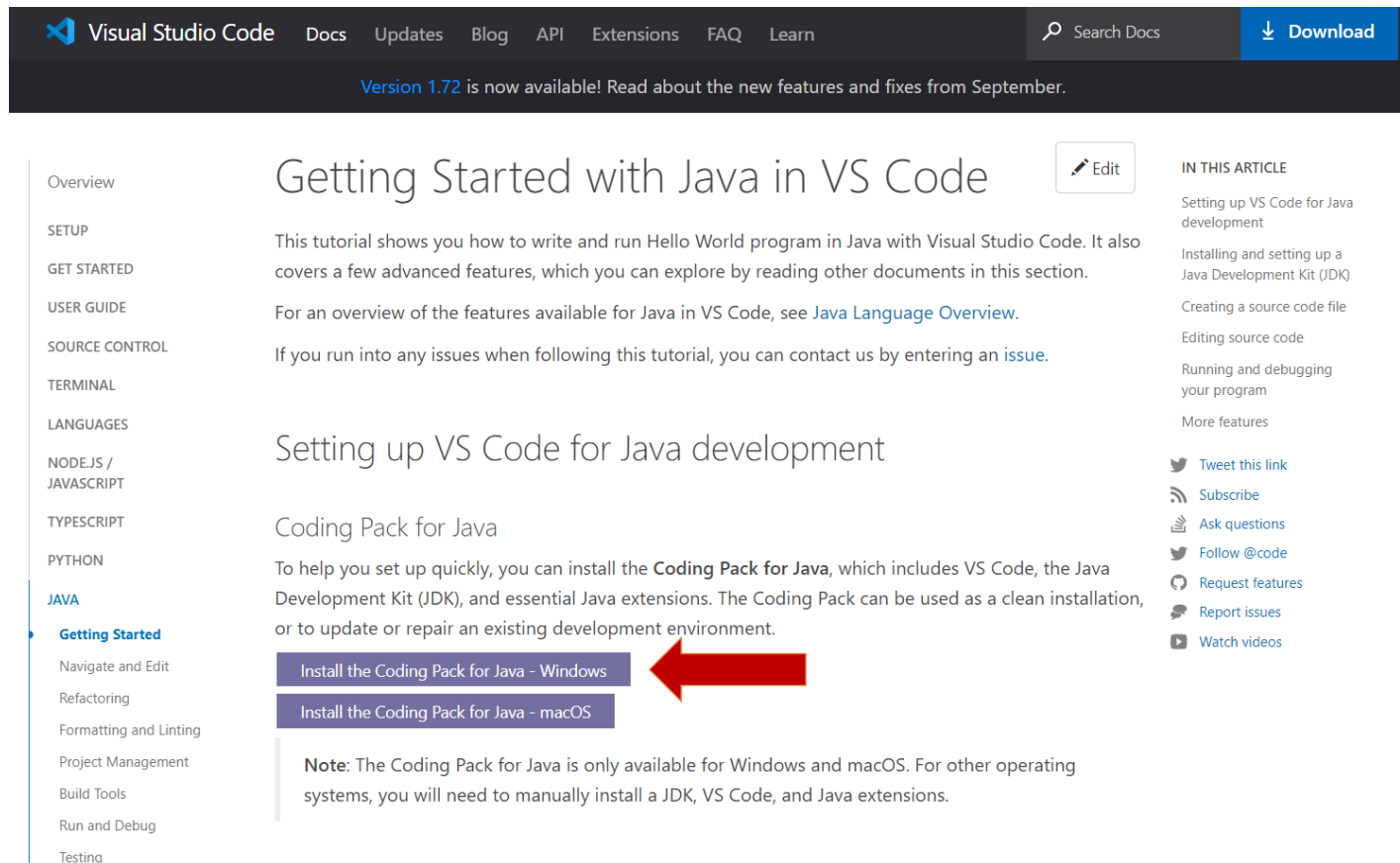
Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.

The screenshot shows the Visual Studio Code download page with three main sections for Windows, Linux, and Mac. Each section has a download button and a list of available download links.

Platform	Download Link	File Size
Windows Windows 8, 10, 11	User Installer	64 bit 62 MB, 32 bit 48 MB
	System Installer	64 bit 62 MB, 32 bit 48 MB
	.zip	64 bit 62 MB, 32 bit 48 MB
	.tar.gz	64 bit 62 MB, 32 bit 48 MB
Linux Debian, Ubuntu	.deb	64 bit 62 MB, 32 bit 48 MB
	.rpm	64 bit 62 MB, 32 bit 48 MB
	.tar.gz	64 bit 62 MB, 32 bit 48 MB
	.tar.gz	64 bit 62 MB, 32 bit 48 MB
Mac macOS 10.11+	.zip	Universal 62 MB, Intel Chip 48 MB, Apple Silicon 48 MB
	.zip	Universal 62 MB, Intel Chip 48 MB, Apple Silicon 48 MB
	.zip	Universal 62 MB, Intel Chip 48 MB, Apple Silicon 48 MB
	.zip	Universal 62 MB, Intel Chip 48 MB, Apple Silicon 48 MB

- Other approach is to install the VS Coding Pack for Java, available at: <https://code.visualstudio.com/docs/java/java-tutorial>



The screenshot shows the Visual Studio Code documentation website. The top navigation bar includes links for Visual Studio Code, Docs, Updates, Blog, API, Extensions, FAQ, and Learn. A search bar and a 'Download' button are also present. A banner below the navigation bar states 'Version 1.72 is now available! Read about the new features and fixes from September.' The main content area is titled 'Getting Started with Java in VS Code' and includes an 'Edit' button. The text describes how to write and run a Hello World program in Java with VS Code. A sidebar on the left lists various topics, with 'Getting Started' highlighted under the 'JAVA' section. Below the main text, there are two buttons: 'Install the Coding Pack for Java - Windows' and 'Install the Coding Pack for Java - macOS'. A red arrow points to the 'Windows' button. A 'Note' at the bottom states that the Coding Pack for Java is only available for Windows and macOS.

Visual Studio Code Docs Updates Blog API Extensions FAQ Learn

Version 1.72 is now available! Read about the new features and fixes from September.

Getting Started with Java in VS Code

This tutorial shows you how to write and run Hello World program in Java with Visual Studio Code. It also covers a few advanced features, which you can explore by reading other documents in this section.

For an overview of the features available for Java in VS Code, see [Java Language Overview](#).

If you run into any issues when following this tutorial, you can contact us by entering an [issue](#).

Setting up VS Code for Java development

Coding Pack for Java

To help you set up quickly, you can install the **Coding Pack for Java**, which includes VS Code, the Java Development Kit (JDK), and essential Java extensions. The Coding Pack can be used as a clean installation, or to update or repair an existing development environment.

Note: The Coding Pack for Java is only available for Windows and macOS. For other operating systems, you will need to manually install a JDK, VS Code, and Java extensions.

Install the Coding Pack for Java - Windows

Install the Coding Pack for Java - macOS

Including:

- VS Code
- Java Development Kit (JDK), and
- essential Java extensions

It is recommended to use this approach, for simplification purpose

SOFTWARE INSTALLATION



Download & run:

1

2

3

4

5

In this case, the VS code is already installed, but the JDK not...

It can happen the other way around...

Or even needed to install both!

Coding Pack for Java

Today (1)

- ☒ JavaCodingPack-0.4.1

Install Coding Pack for Java

This Setup Wizard will automatically download the JDK distribution on your behalf.

The JDK binary to be installed is provided by [licensed](#). By downloading and using Visual Studio Code, you agree to the [license terms](#). The Microsoft and third party components are licensed under these [licenses](#). The third party components are licensed under the accompanying licenses.

By continuing, you agree to these licenses.

☒ I accept the terms in the License Agreement

Configure Components

Required components are listed below. Click Install to proceed.

Java Development Kit: N/A
Visual Studio Code: 1.72.2

Installing Components

Download JDK

Downloading OpenJDK17U-jdk_x64_windows_hotspot_17.0.4.1_1.msi: 80.34M/161.14M

[10:02:51 AM] AdoptOpenJDK-17-x64

Completing the Setup Wizard

The Wizard has finished installing all the components on your computer. The application may be launched by selecting the installed icons.

Click Finish to exit the Wizard.

☒ Launch Visual Studio Code

Finish

Cancel

Next

Cancel

Back

Next

Testing JDK

Run a Command Prompt Window (in the search button, write down “cmd.exe” or press Win + R and write “cmd”)

Inside the prompt, use the command:

and { `java -version` →
`javac -version` →

```
Command Prompt
Microsoft Windows [Version 10.0.22621.2283]
(c) Microsoft Corporation. All rights reserved.

C:\Users\faf>java -version
openjdk version "17.0.4.1" 2022-08-12
OpenJDK Runtime Environment Temurin-17.0.4.1+1 (build 17.0.4.1+1)
OpenJDK 64-Bit Server VM Temurin-17.0.4.1+1 (build 17.0.4.1+1, mixed mode, sharing)

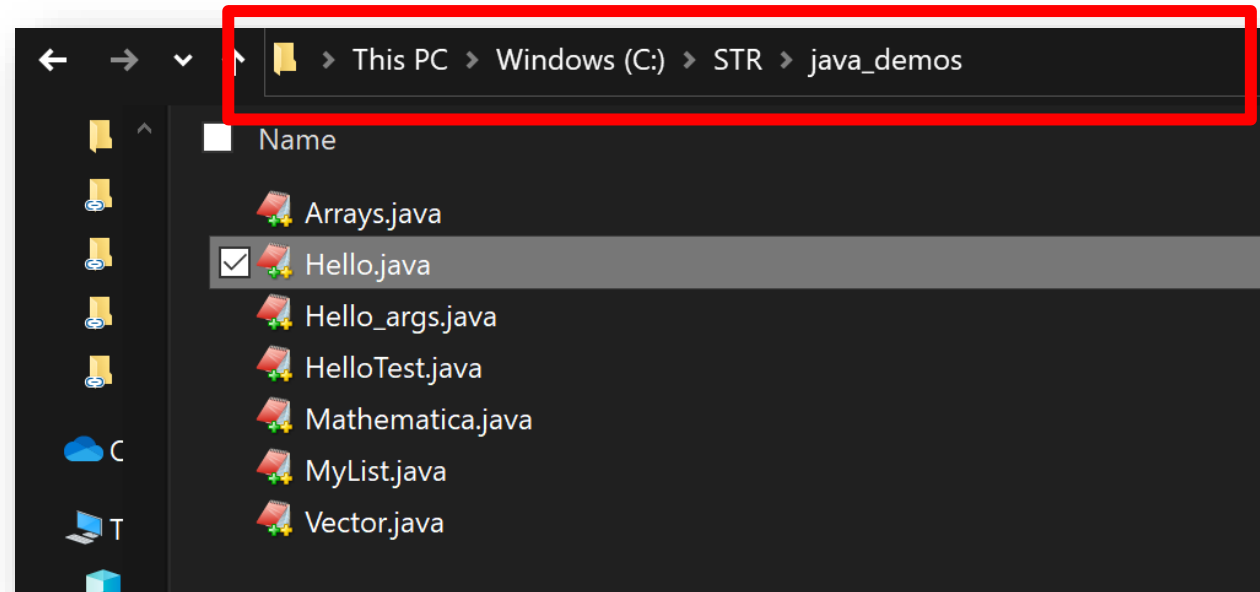
C:\Users\faf>javac -version
javac 17.0.4.1

C:\Users\faf>
```

If the result is similar to the figure, then **SUCCESS**.



Extract java_demos.zip to c:\str



Example 1 – My first Java program

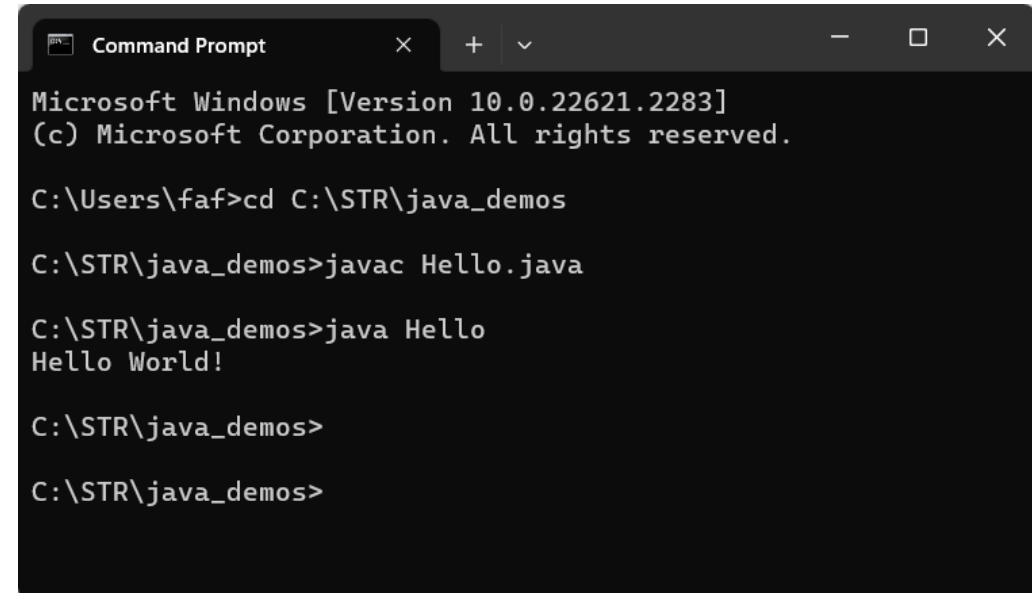
In `c:\str\java_demos` edit the file named **Hello.java**

```
1 class Hello
2 {
3     public static void main(String[] args)
4     {
5         System.out.println("Hello World!");
6     }
7 }
```

Enter a console window and go to `c:\str\java_demos`

Compile with: **javac Hello.java**

Execute with: **java Hello**



```
Command Prompt
Microsoft Windows [Version 10.0.22621.2283]
(c) Microsoft Corporation. All rights reserved.

C:\Users\faf>cd C:\STR\java_demos

C:\STR\java_demos>javac Hello.java

C:\STR\java_demos>java Hello
Hello World!

C:\STR\java_demos>

C:\STR\java_demos>
```

Example 2 – Using arguments in method “main”

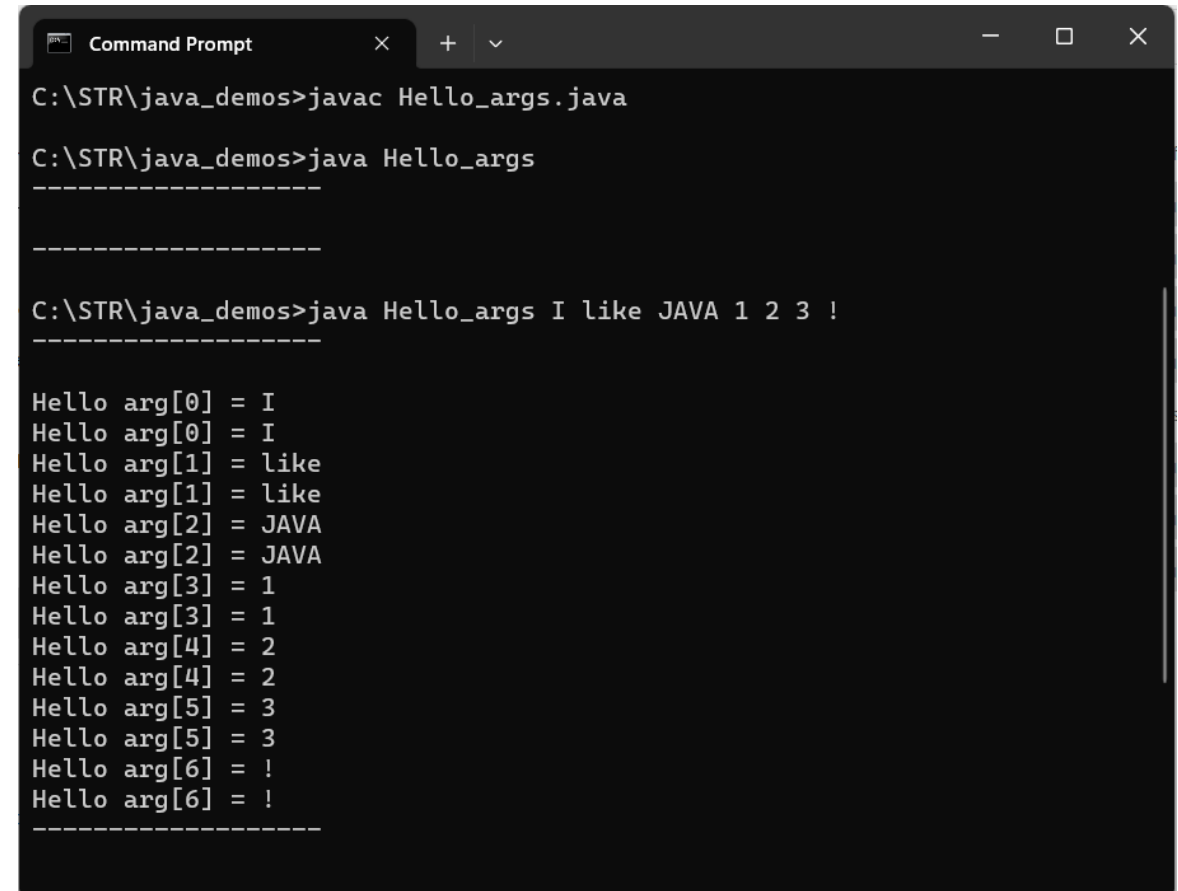
In `c:\str\java_demos` edit the file named `Hello_args.java`

```
1 public class Hello_args
2 {
3     public static void main(String[] args){
4         System.out.println("-----");
5         for (int i=0; i<args.length; i++){
6
7             System.out.printf("\nHello arg[%d] = %s", i ,args[i]);
8
9             System.out.printf("\nHello arg[" + i + "] = " + args[i] );
10
11     }
12     System.out.println("\n-----");
13 }
14 }
```

Compile with: `javac Hello_args.java`

Execute with: `java Hello_args`

and `java Hello_args I like JAVA 1 2 3 !`



```
Command Prompt
C:\STR\java_demos>javac Hello_args.java
C:\STR\java_demos>java Hello_args
-----

C:\STR\java_demos>java Hello_args I like JAVA 1 2 3 !
-----

Hello arg[0] = I
Hello arg[0] = I
Hello arg[1] = like
Hello arg[1] = like
Hello arg[2] = JAVA
Hello arg[2] = JAVA
Hello arg[3] = 1
Hello arg[3] = 1
Hello arg[4] = 2
Hello arg[4] = 2
Hello arg[5] = 3
Hello arg[5] = 3
Hello arg[6] = !
Hello arg[6] = !
-----
```

Example 3 – Using math functions

In `c:\str\java_demos` edit the file named **Vector.java**

The code at the right implements a class vector, which is not a program, but a class for defining the mathematical concept of vector.

```
1 public class Vector {
2     private double x;
3     private double y;
4     private double z;
5
6     public Vector(double x, double y, double z) {
7         this.x = x;
8         this.y = y;
9         this.z = z;
10    }
11    public void sum(double x, double y, double z){
12        this.x+= x; // this.x = this.x + x;
13        this.y+= y;
14        this.z+= z;
15    }
16    public Vector scalarProduct(Vector v) {
17        Vector result = new Vector(0,0,0);
18        result.x = this.x * v.x;
19        result.y = this.y * v.y;
20        result.z = this.z * v.z;
21        return result;
22    }
23    public double module() {
24        double module = Math.sqrt( Math.pow(this.x,2)+Math.pow(this.y,2)+Math.pow(this.z,2) );
25        return module;
26    }
27    public String toString() {
28        String s = new String(" "+this.x+"x "+this.y+"y "+this.z+"z");
29        return s;
30    }
31 }
32
```

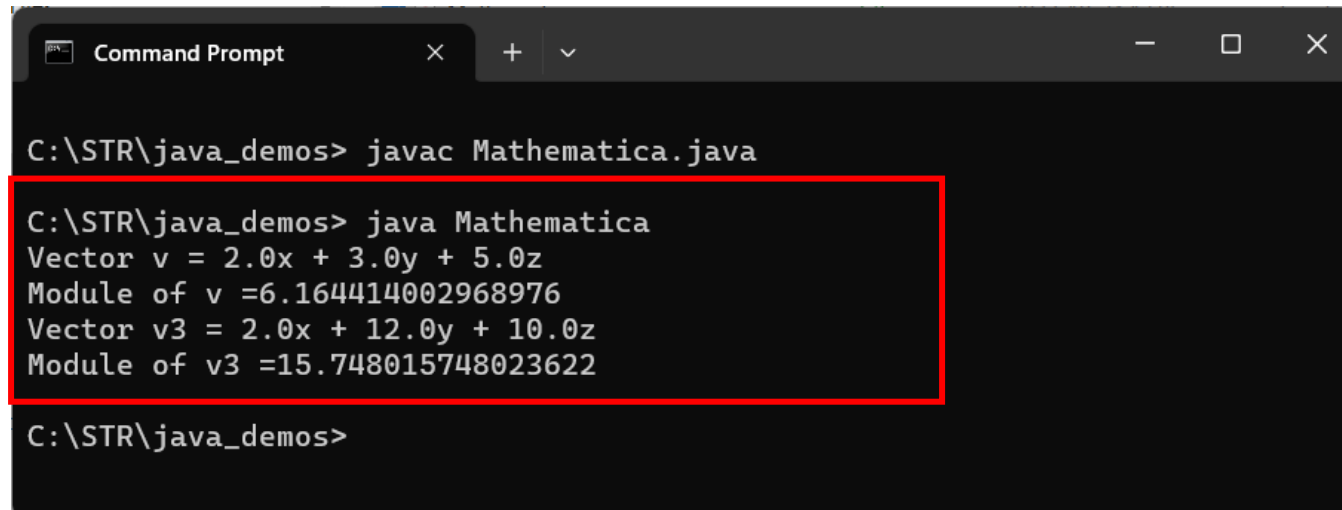
Example 3 – Using math functions

In `c:\str\java_demos` edit the file named **Mathematica.java**

```
1 public class Mathematica {  
2  
3     public Mathematica() {  
4         System.out.println("Put initialization code here");  
5     }  
6     static public void main(String args[]) {  
7  
8         Vector v = new Vector(2,3,5);  
9  
10        System.out.println("Vector v =" + v.toString());  
11        System.out.println("Module of v =" + v.module());  
12  
13        Vector v2 = new Vector(1,4,2);  
14        Vector v3 = v.scalarProduct(v2);  
15  
16        System.out.println("Vector v3 =" + v3);  
17        System.out.println("Module of v3 =" + v3.module() );  
18    }  
19 }  
20
```

It uses the code of class `Vector` for performing operations with vectors.

Modularity is one of the great OOP features existing in languages like JAVA.



```
Command Prompt  
C:\STR\java_demos> javac Mathematica.java  
C:\STR\java_demos> java Mathematica  
Vector v = 2.0x + 3.0y + 5.0z  
Module of v =6.164414002968976  
Vector v3 = 2.0x + 12.0y + 10.0z  
Module of v3 =15.748015748023622  
C:\STR\java_demos>
```

Example 4 – Using arrays

In `c:\str\java_demos` edit the file named `Arrays.java`

```
Command Prompt
C:\STR\java_demos> javac Arrays.java

C:\STR\java_demos> java Arrays
what , is , the , matrix , ?
you , must , see , it , yourself , ...
matrix={
  a , f , 1 , b
  d , 5 , 2 , 8
  1 , 3 , 9 , e
  f , a , c , a
}

C:\STR\java_demos>
```

```
1 public class Arrays {
2
3     public void printArray(Object array[]){
4         for(int i=0; i<array.length; i++) {
5             if(i>0)
6                 System.out.print(" ");
7                 System.out.print(array[i]);
8             }
9             System.out.println();
10    }
11
12    public void printMatrix(Object matrix[][] ) {
13        System.out.println("matrix=");
14        for(int i=0; i<matrix.length; i++) {
15            System.out.print(" ");
16            printArray( matrix[i] );
17        }
18        System.out.println("}");
19    }
20
21    static public void main(String args[]) {
22        String array1[]={"what ", "is ", "the ", "matrix", "?"};
23        String array2[]={"you ", "must ", "see ", "it ", "yourself", "..."};
24        String matrix[][] = {
25            {"a", "f", "1", "b"},
26            {"d", "5", "2", "8"},
27            {"1", "3", "9", "e"},
28            {"f", "a", "c", "a"}
29        };
30        Arrays d = new Arrays();
31        d.printArray(array1);
32        d.printArray(array2);
33        d.printMatrix(matrix);
34    }
35 }
```

Example 5 – Using linked lists

In `c:\str\java_demos` edit the file named `MyList.java`

```
Command Prompt
C:\STR\java_demos>javac MyList.java

C:\STR\java_demos>java MyList
Object of MyList just created!
-----
first(list)=aaa
last (list)=zzz
-----
list[0]=aaa
list[1]=ccc
list[2]=222
list[3]=100
list[4]=zzz
-----

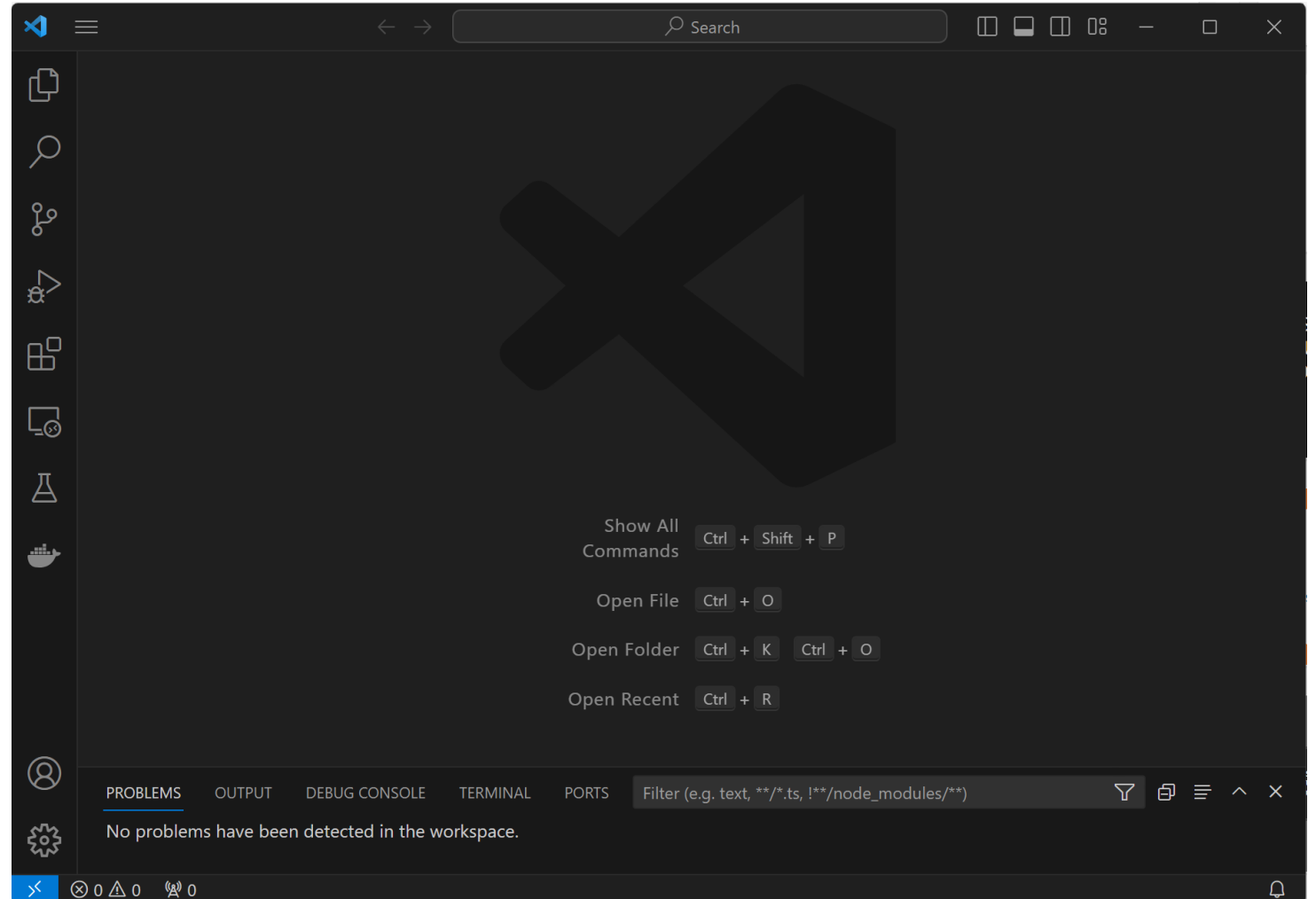
C:\STR\java_demos>
```

```
1 public class MyList {
2
3     public MyList() {
4         System.out.println("Object of MyList just created...");
5     }
6     static public void main(String args[]) {
7         java.util.LinkedList list = new java.util.LinkedList();
8
9         list.add("aaa");
10        list.add("ccc");
11        list.add("222");
12        list.add( new Integer(100) );
13        list.add("zzz");
14
15        System.out.println("-----");
16        System.out.println( "first(list)=" +list.getFirst());
17        System.out.println( "last (list)=" +list.getLast());
18        System.out.println("-----");
19
20        for(int i=0; i<list.size(); i++) {
21            System.out.println( "list["+i+"]="+list.get(i) );
22        }
23        System.out.println("-----");
24    }
25 }
```


VISUAL STUDIO CODE



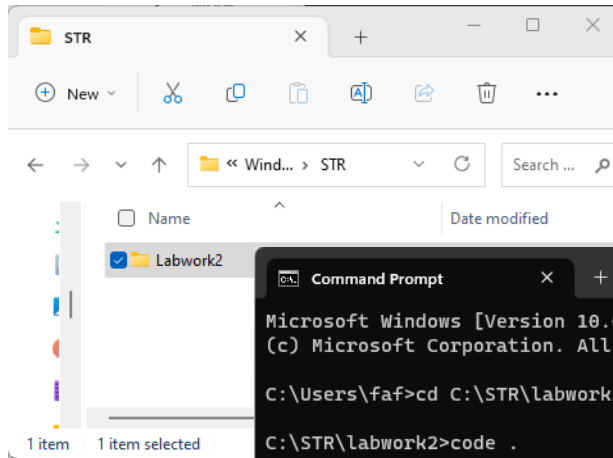
Visual Studio Code



VS CODE: CREATE A NEW PROJECT

There are **two ways** for creating a new project in VS Code:

1. a) Create a new folder in c:/STR named **Labwork2**

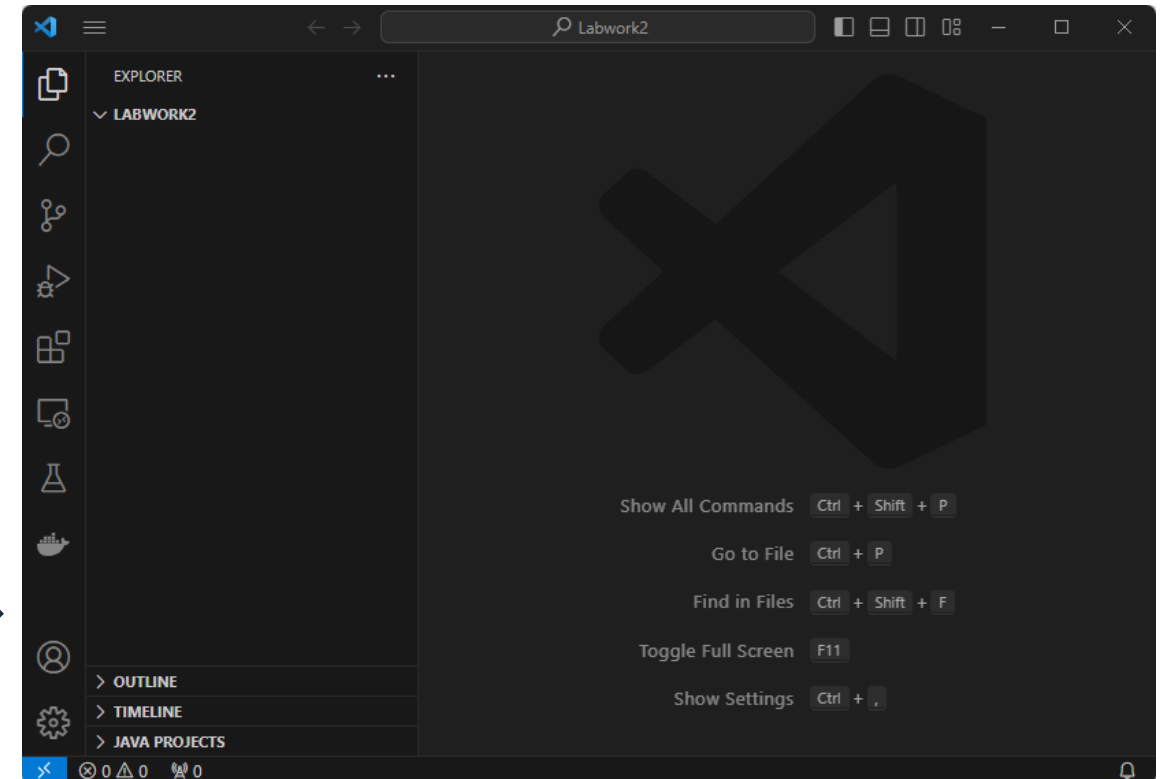


```
Microsoft Windows [Version 10.0.22621.2428]
(c) Microsoft Corporation. All rights reserved.

C:\Users\faf>cd C:\STR\labwork2

C:\STR\labwork2>code .

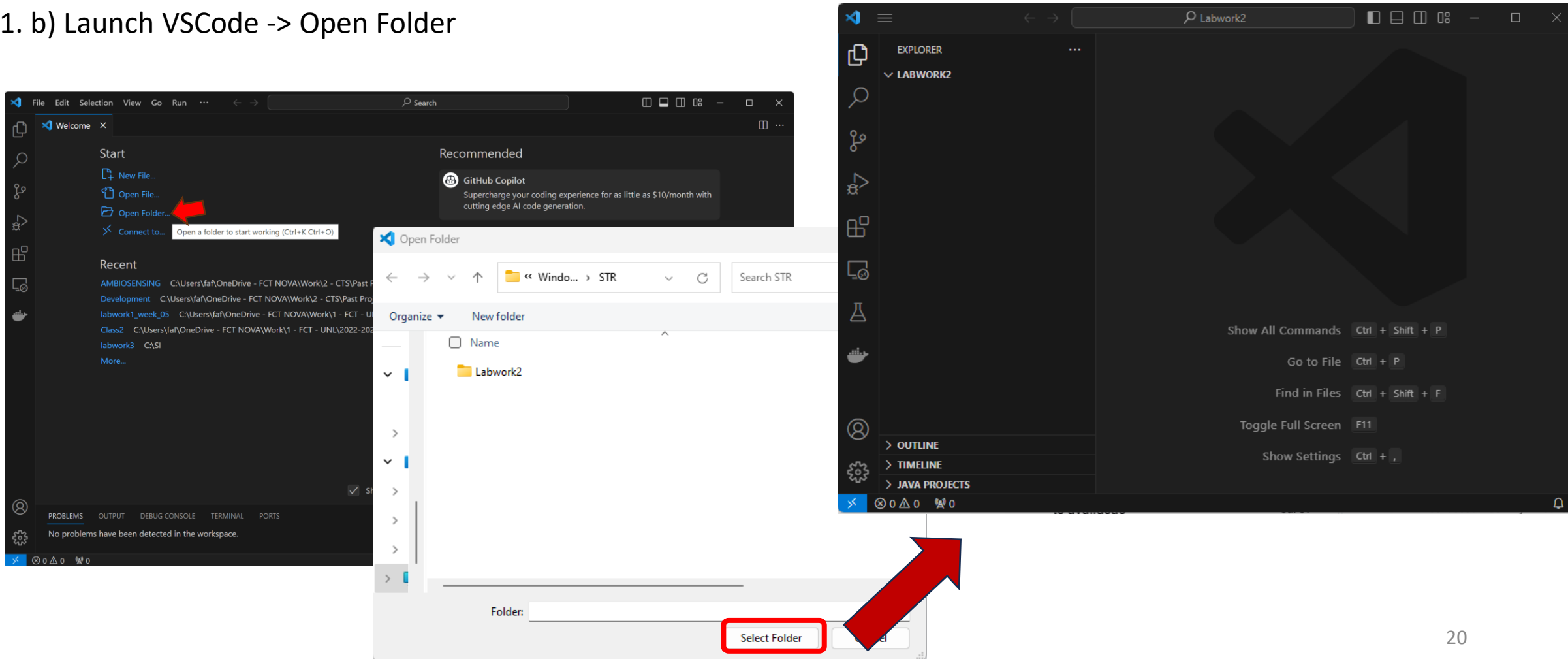
C:\STR\labwork2>
```



VS CODE: CREATE A NEW PROJECT

There are **two ways** for creating a new project in VS Code:

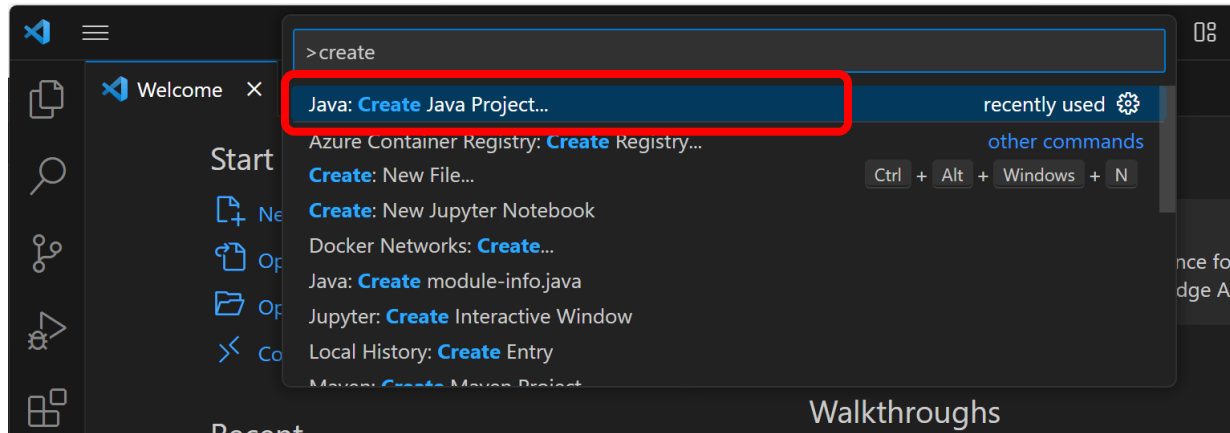
1. b) Launch VSCode -> Open Folder



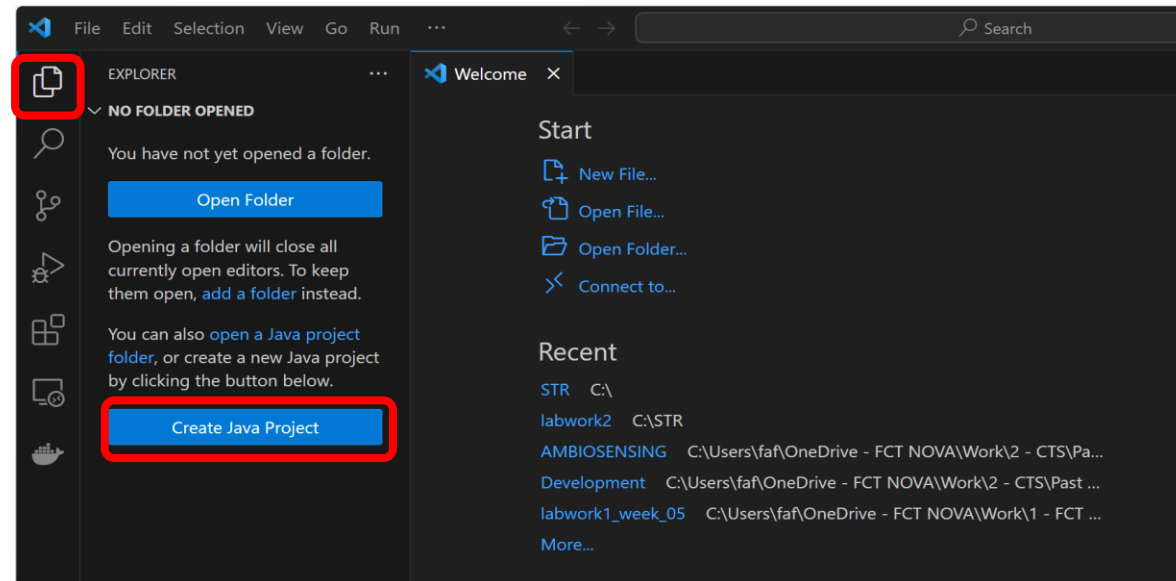
VS CODE: CREATE A NEW PROJECT

VS Code does **not** include a traditional **File > New** Project dialog or pre-installed project templates. There are **two ways** for creating a new JAVA project in VS Code:

2. a) Press F1 -> write “create” -> select “Create Java Project”



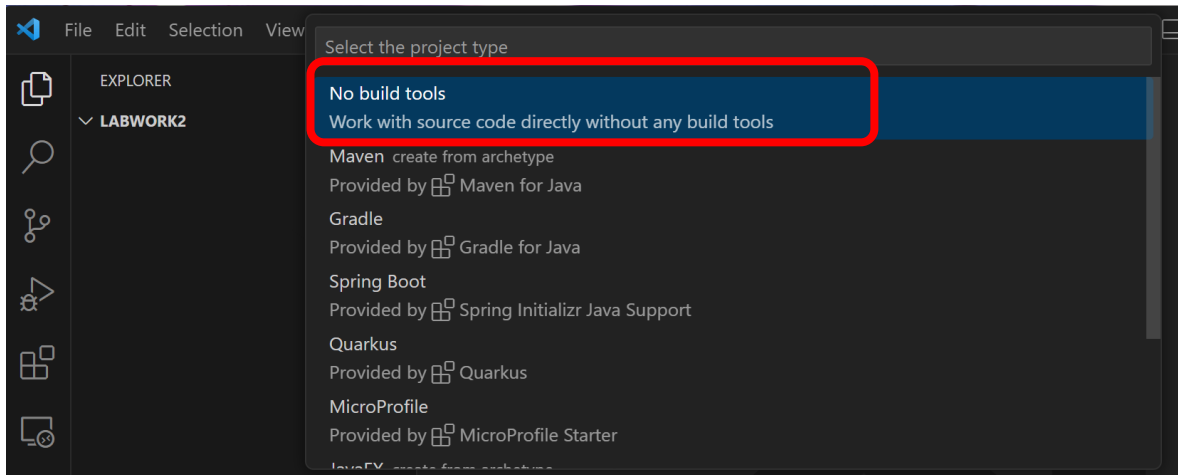
2. b) Click on the “Explorer” icon on the left-hand menu and click on the “Create Java Project” button



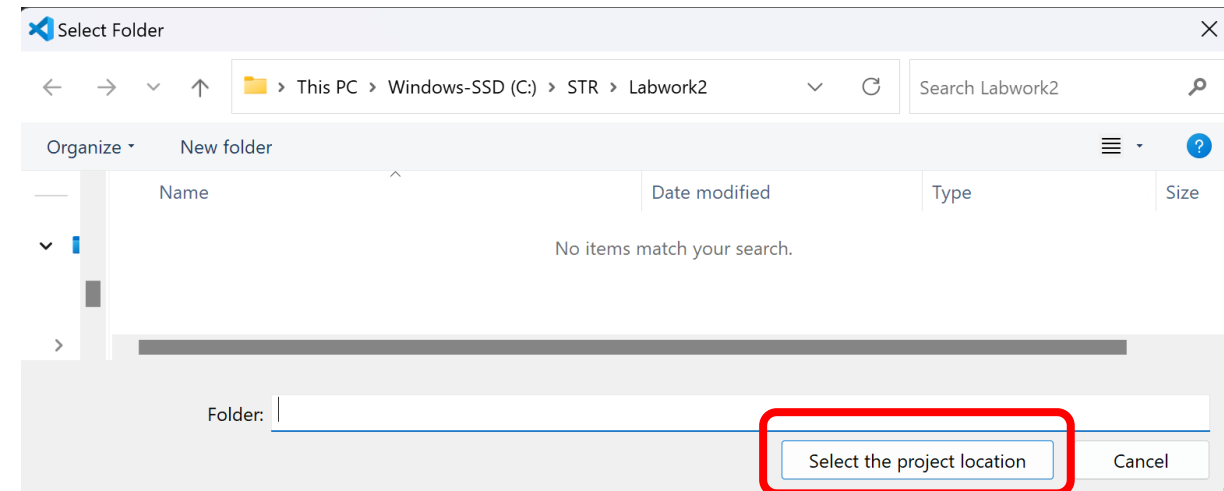
VS CODE: CREATE A NEW PROJECT



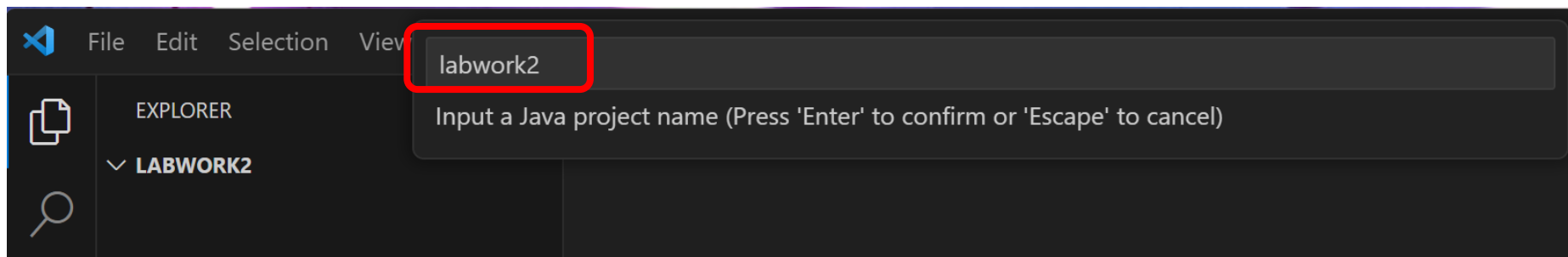
3. Select “No build tools”



4. Select project folder (already selected...)



5. Insert “labwork2” as the project’s name



VS CODE: CREATE A NEW PROJECT



6. Project "labwork2" created -> run for testing success

The screenshot shows the Visual Studio Code interface with a project named "labwork2". The Explorer sidebar on the left shows the project structure with folders ".vscode", "bin", "lib", and "src", and a file "App.java" selected. The main editor displays the code for "App.java":

```
src > J App.java > App
1 public class App {
2     public static void main(String[] args) throws Exception {
3         System.out.println(x:"Hello, World!");
4     }
5 }
6
```

The "Run" button (a play icon) in the top right corner of the editor is highlighted with a red box. Below the editor, the TERMINAL panel shows the output of the program:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\STR\Labwork2\labwork2> & 'C:\Users\faf\AppData\Local\Programs\Eclipse Adoptium\jdk-17.0.4.101-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\STR\Labwork2\labwork2\bin' 'App'
Hello, World!
PS C:\STR\Labwork2\labwork2>
```

The output "Hello, World!" is highlighted with a red box. The status bar at the bottom indicates "Ln 1, Col 1", "Spaces: 4", "UTF-8", "LF", and "Java".



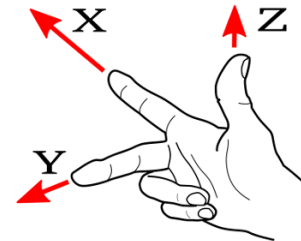
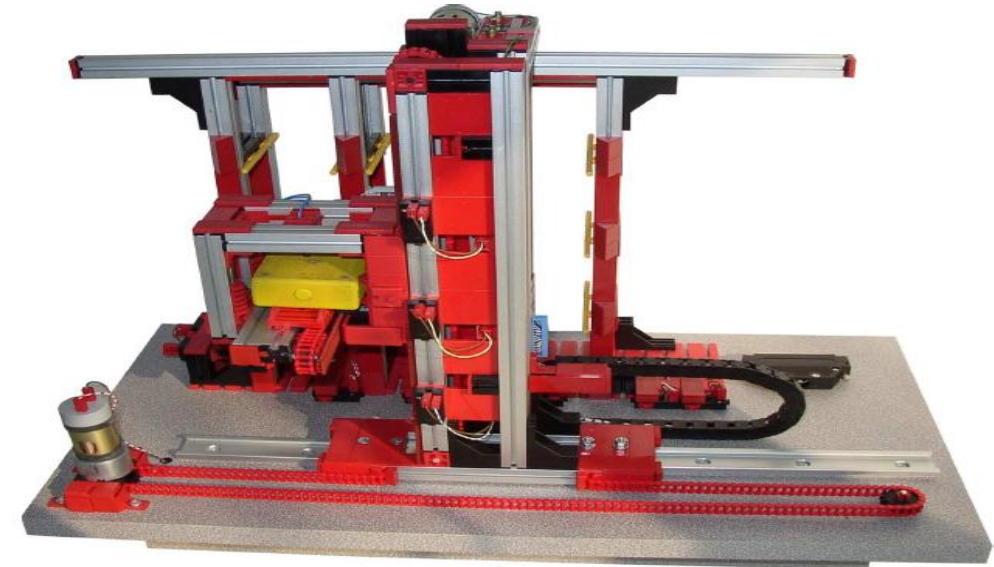
PART 2

Interaction with the Physical System

INTRODUCING THE “STORAGE” KIT

The physical storage kit

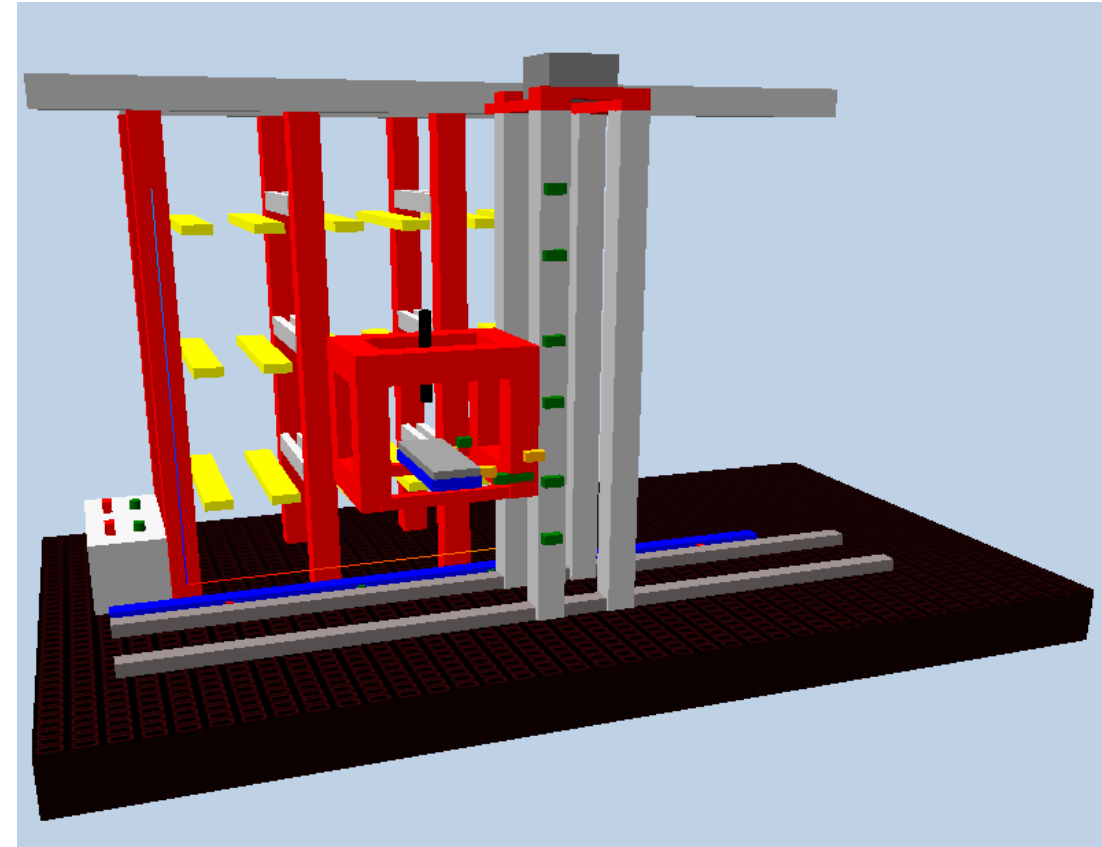
- ❑ Here is a picture of the storage kit.
- ❑ We can move the cage with a pallet on the xx , yy , zz axis with directions according to the right-hand rule.
- ❑ Pallets put or taken from the cells.



<https://www.ias.informatik.tu-darmstadt.de/Teaching/RobotLearningLecture2>

Using the Simulator

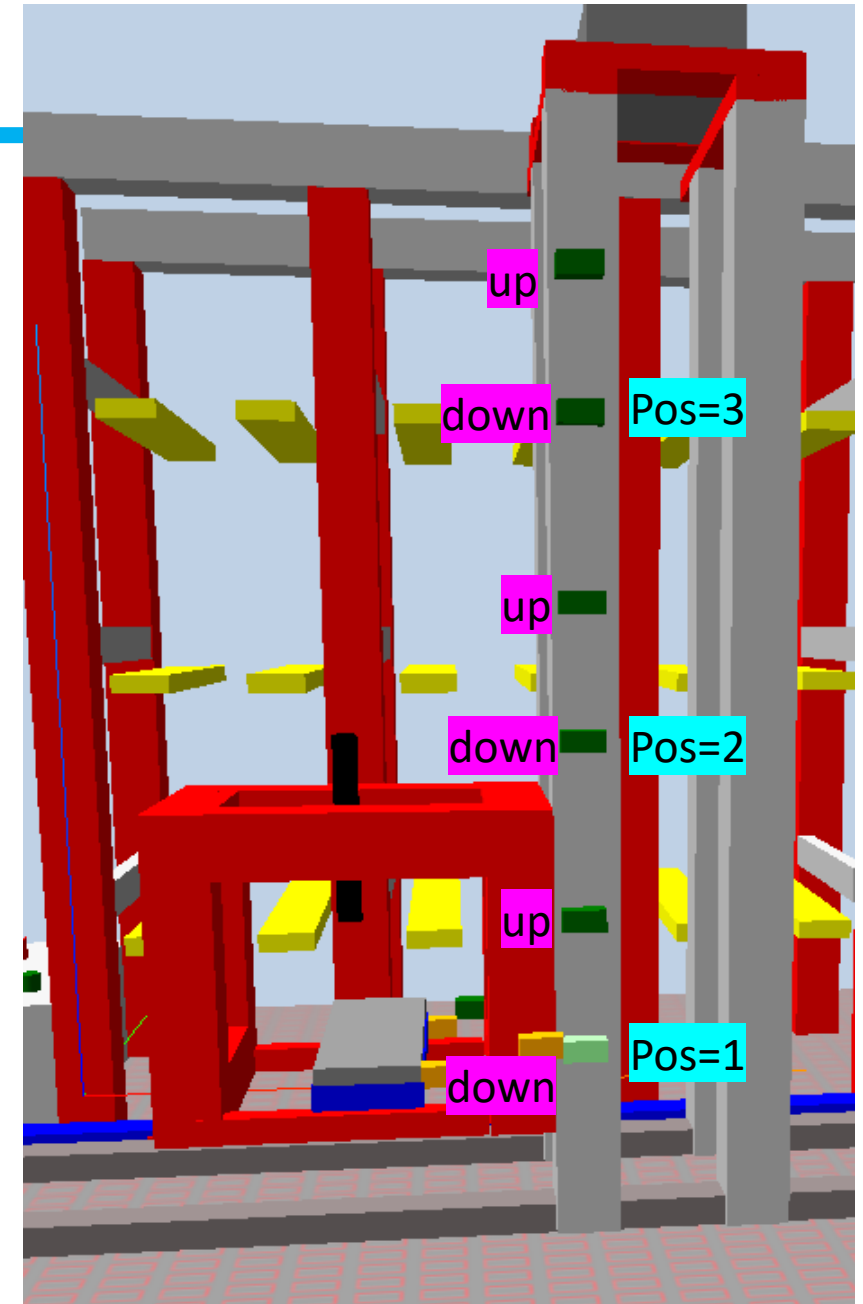
- ☐ There is a simulator to replace the physical storage kit to overcome remote classes context (sometimes, even in classroom, due to the number of students)
- ☐ All functionality and coding errors can be solved using the simulator.
- ☐ Switching from kit to simulator could be done without any changes in your C++ code.



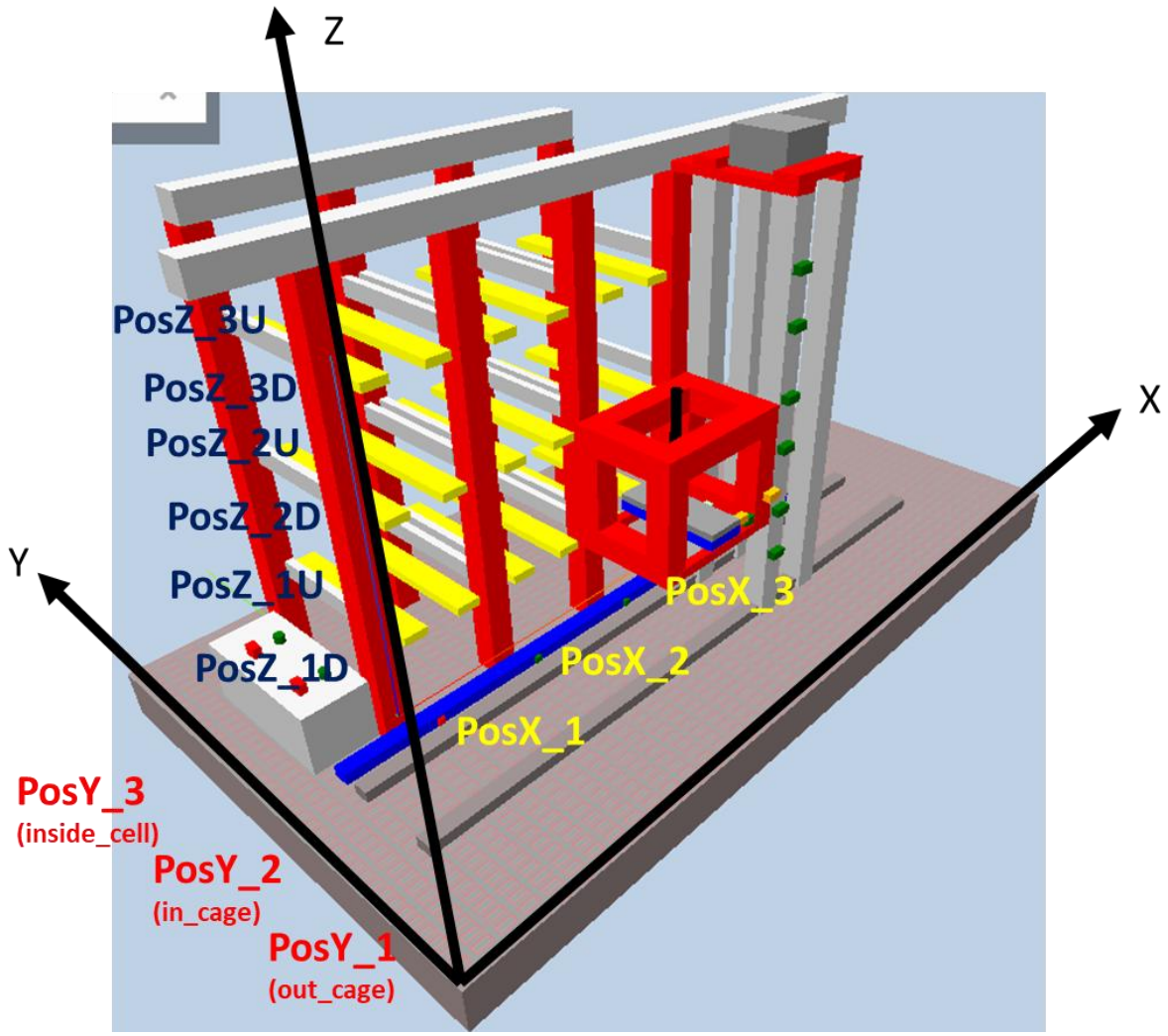
INTRODUCING THE “STORAGE” KIT

Z Axis particularities

- ☐ Z axis has 3 positions.
- ☐ Each position has a pair of sensors: up_position, down_position
- ☐ Use the down_positions **ONLY** for the vertical **position 1,2 and 3**.
- ☐ up_position and down_position sensors are used in
putPartInCell and *takePartFromCell* (to see later)



INTRODUCING THE “STORAGE” KIT

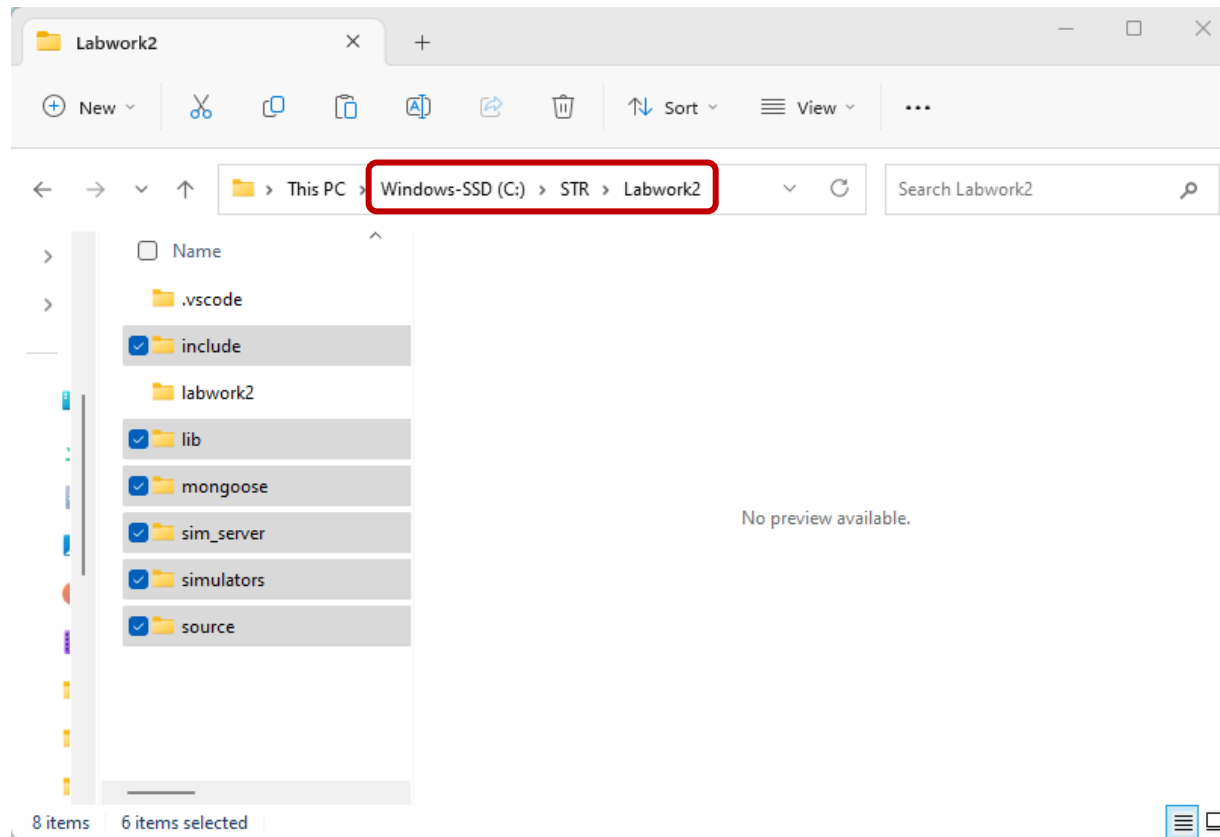


Axis	Position	Tuple p<port.bit>	OBS.
X	PosX_1	P0.2	active low
	PosX_2	P0.1	
	PosX_3	P0.0	
Y	PosY_1	P0.5	
	PosY_2	P0.4	
	PosY_3	P0.3	
Z	PosZ_1D	P1.3	
	PosZ_1U	P1.2	
	PosZ_2D	P1.1	
	PosZ_2U	P1.0	
	PosZ_3D	P0.7	
	PosZ_3U	P0.6	

LABWORK2 PROJECT PACKAGE



- ❑ Copy the material for Labwork from CLIP
- ❑ Extract STR_TP2_Storage_Base_Package.zip into `c:\str\Labwork2`, as shown in the picture below.

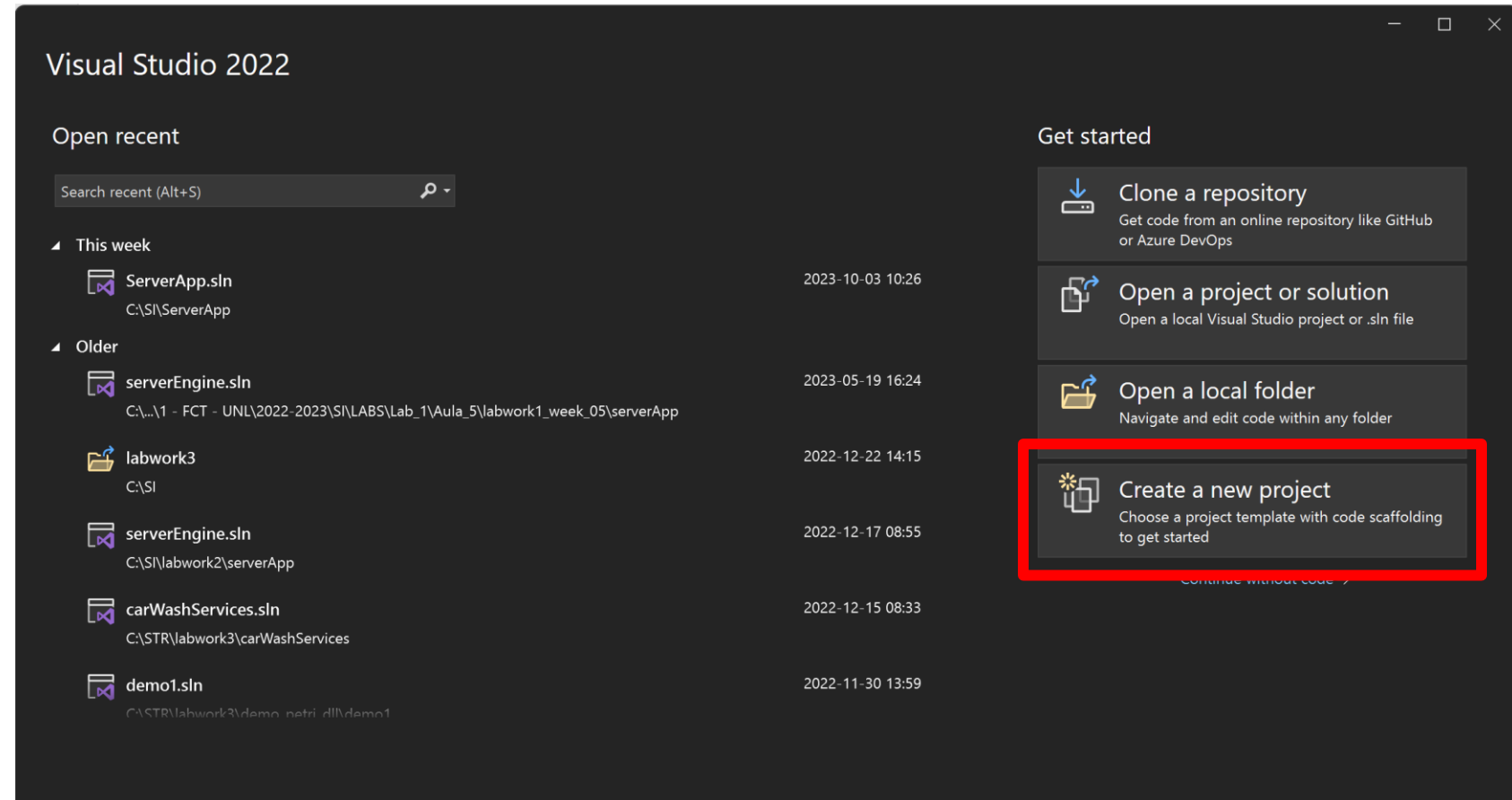


(*) please comply with directory, as it allows faster evaluation of your labworks.

CREATION OF VS CONSOLE APPLICATION PROJECT



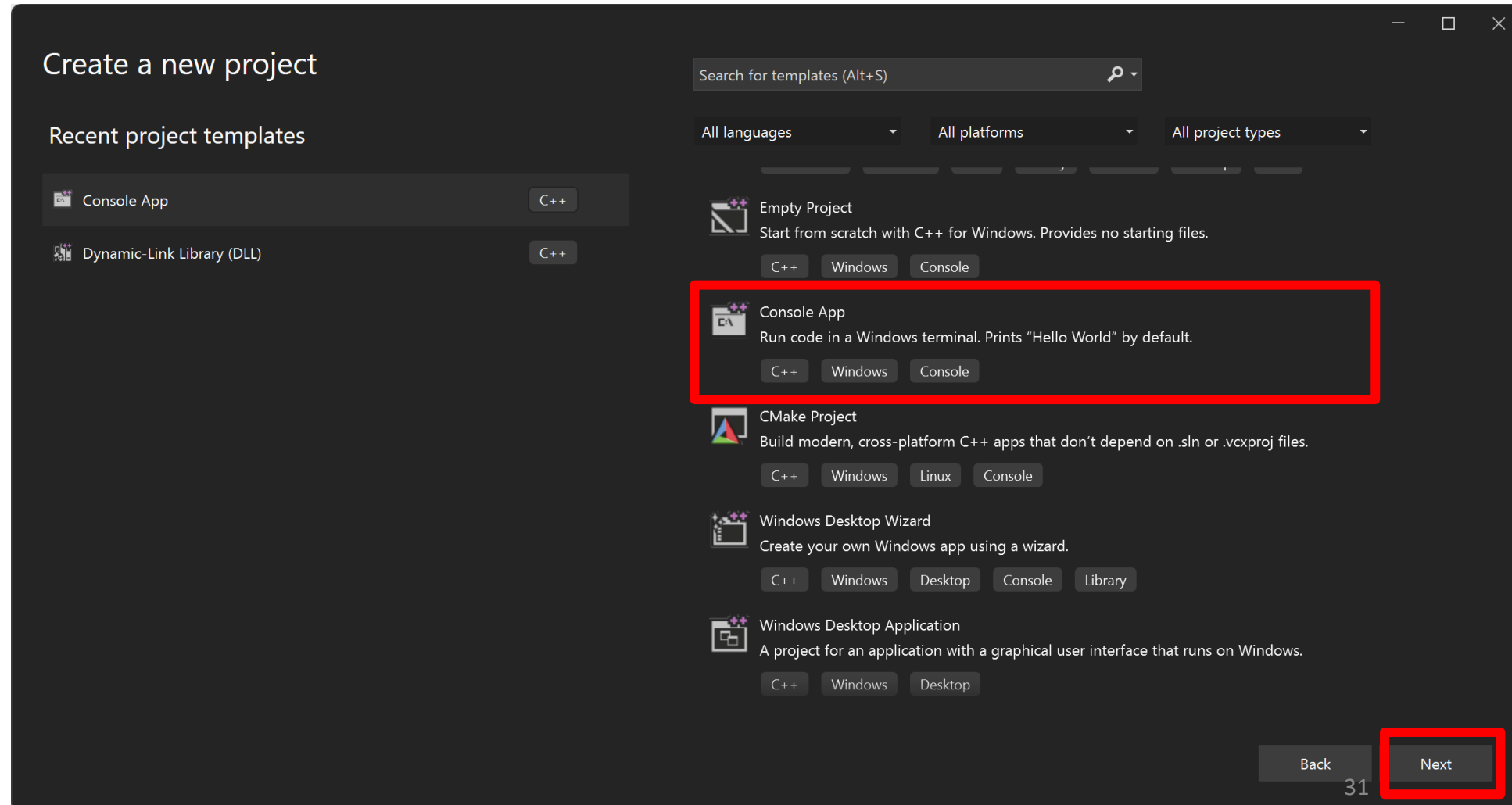
- ❑ Find Visual Studio in your startup menu -> Run it as administrator
- ❑ Next steps are for project creation and configuration



CREATION OF VS CONSOLE APPLICATION PROJECT



Select Console Application



CREATION OF VS CONSOLE APPLICATION PROJECT



- ☐ Project name: **storage**
- ☐ Location: **c:\str\Labwork2**
- ☐ Click on the checkbox

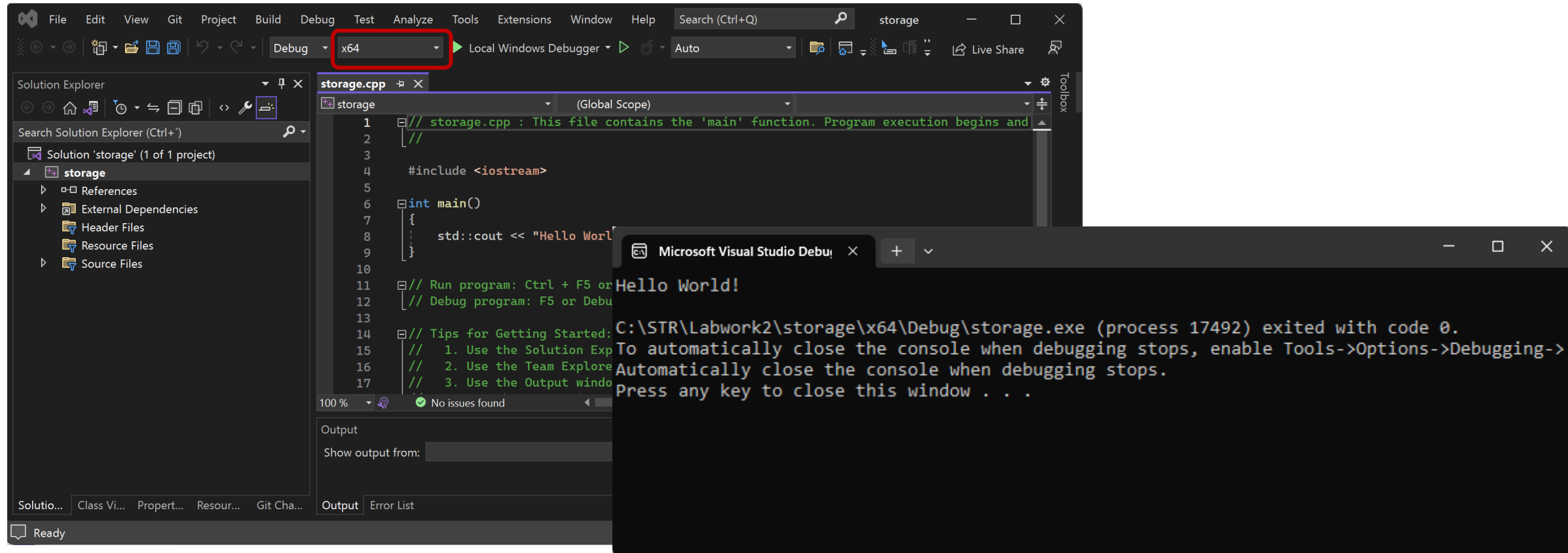
A screenshot of the Visual Studio 'Configure your new project' dialog box. The dialog has a dark theme. At the top, it says 'Configure your new project'. Below that, there are three tabs: 'Console App', 'C++', 'Windows', and 'Console'. The 'Console App' tab is selected. The 'Project name' field contains 'storage'. The 'Location' field contains 'C:\STR\Labwork2\'. The 'Solution name' field contains 'storage'. There is a checkbox labeled 'Place solution and project in the same directory' which is checked. At the bottom right, there are two buttons: 'Back' and 'Create'. The 'Create' button is highlighted with a red rectangle. Several other fields and the checkbox are also highlighted with red rectangles.

CREATION OF VS CONSOLE APPLICATION PROJECT



Verification:

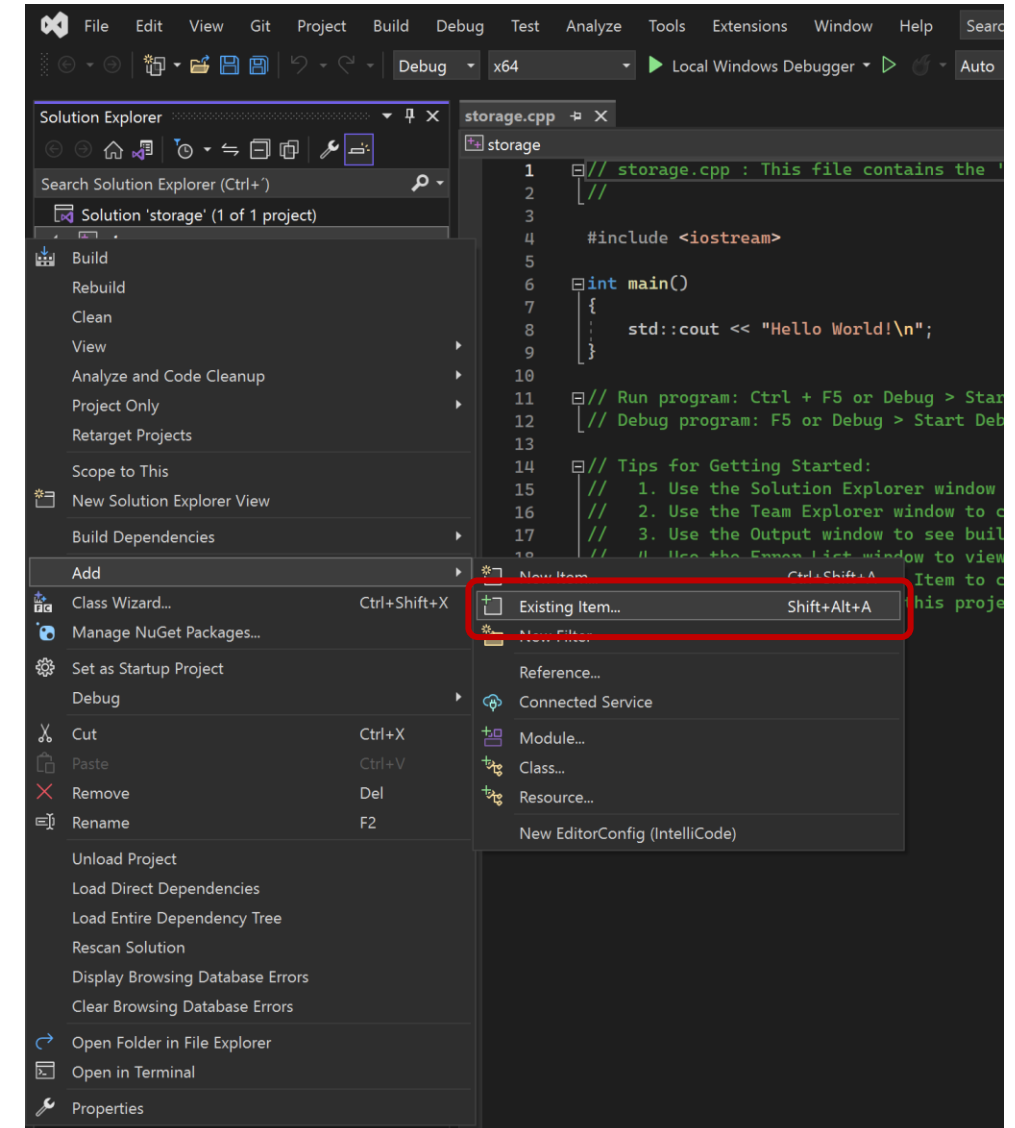
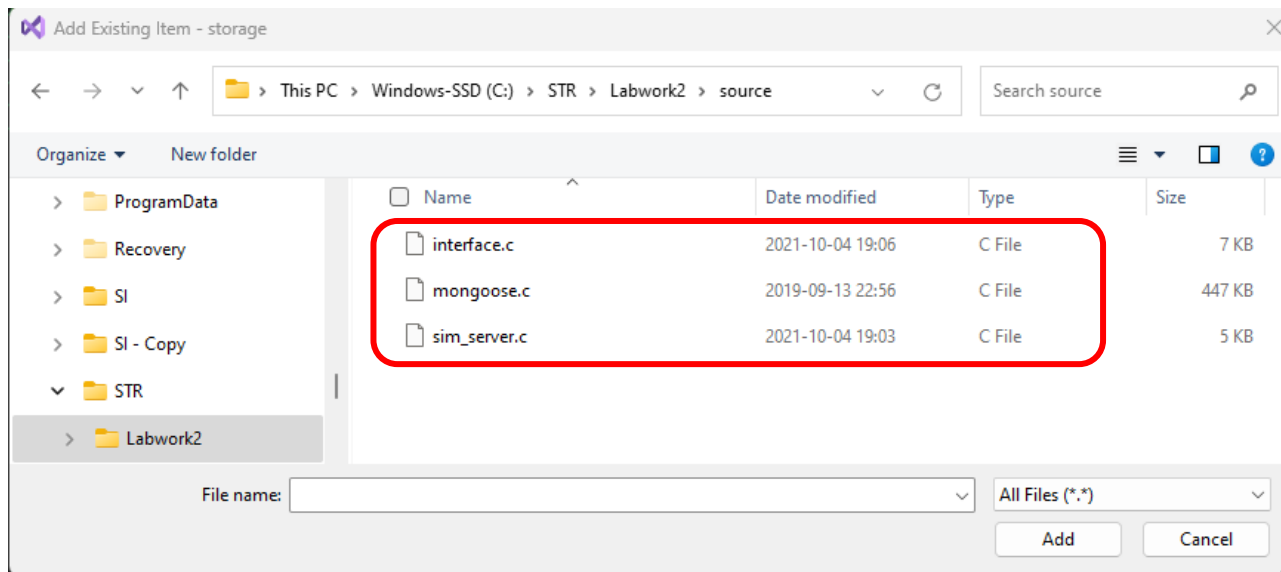
- ☐ Set the solution **platform to x64** bit
- ☐ **Run** the project to check if it is ok until now.



CONFIGURE PROJECT PROPERTIES



- ❑ Right-click on project and select **Add > Existing item**
- ❑ Select *.c files from **c:\str\Labwork2\source** folder and **add** to project

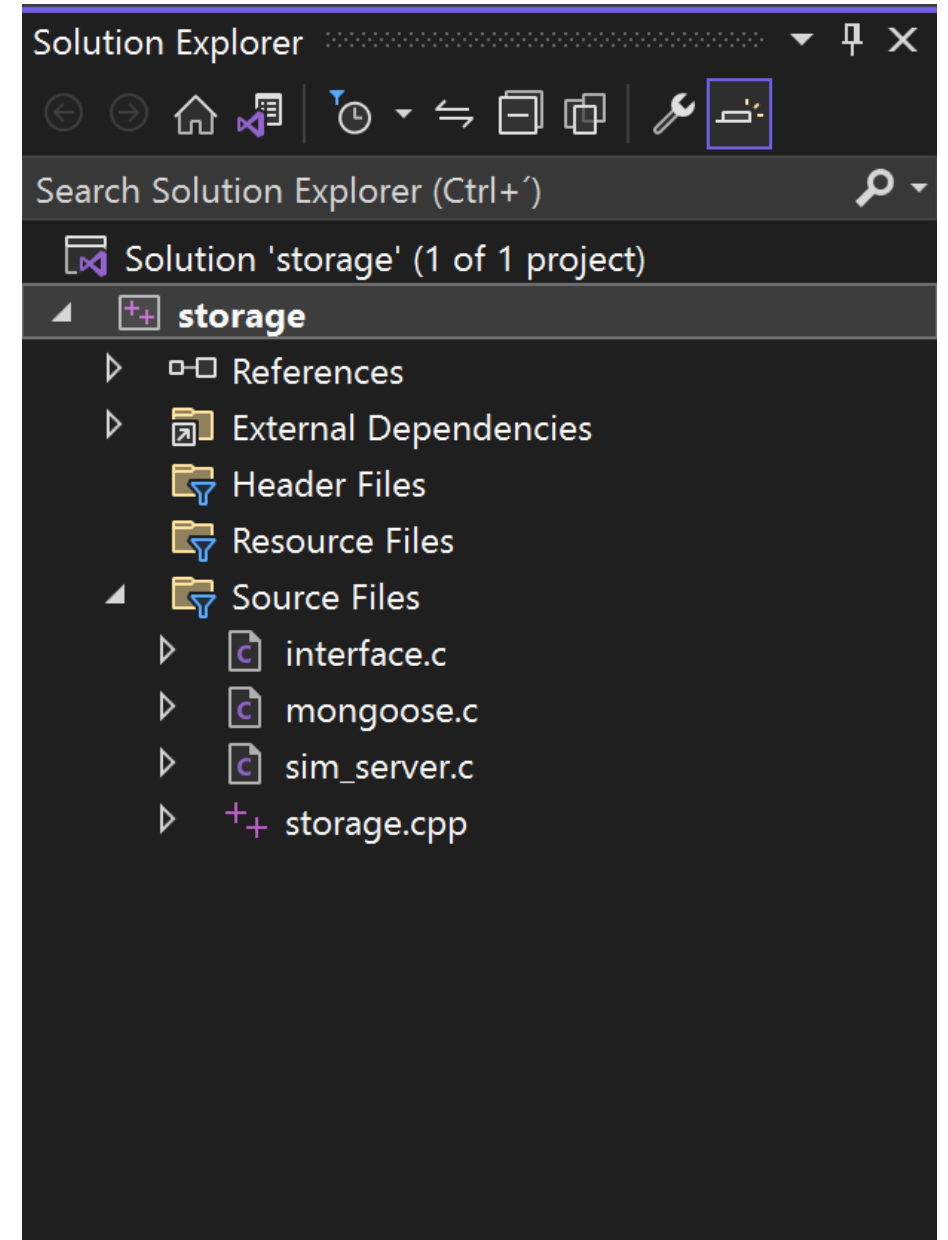


CONFIGURE PROJECT PROPERTIES



Aspect of your solution explorer in Visual Studio:

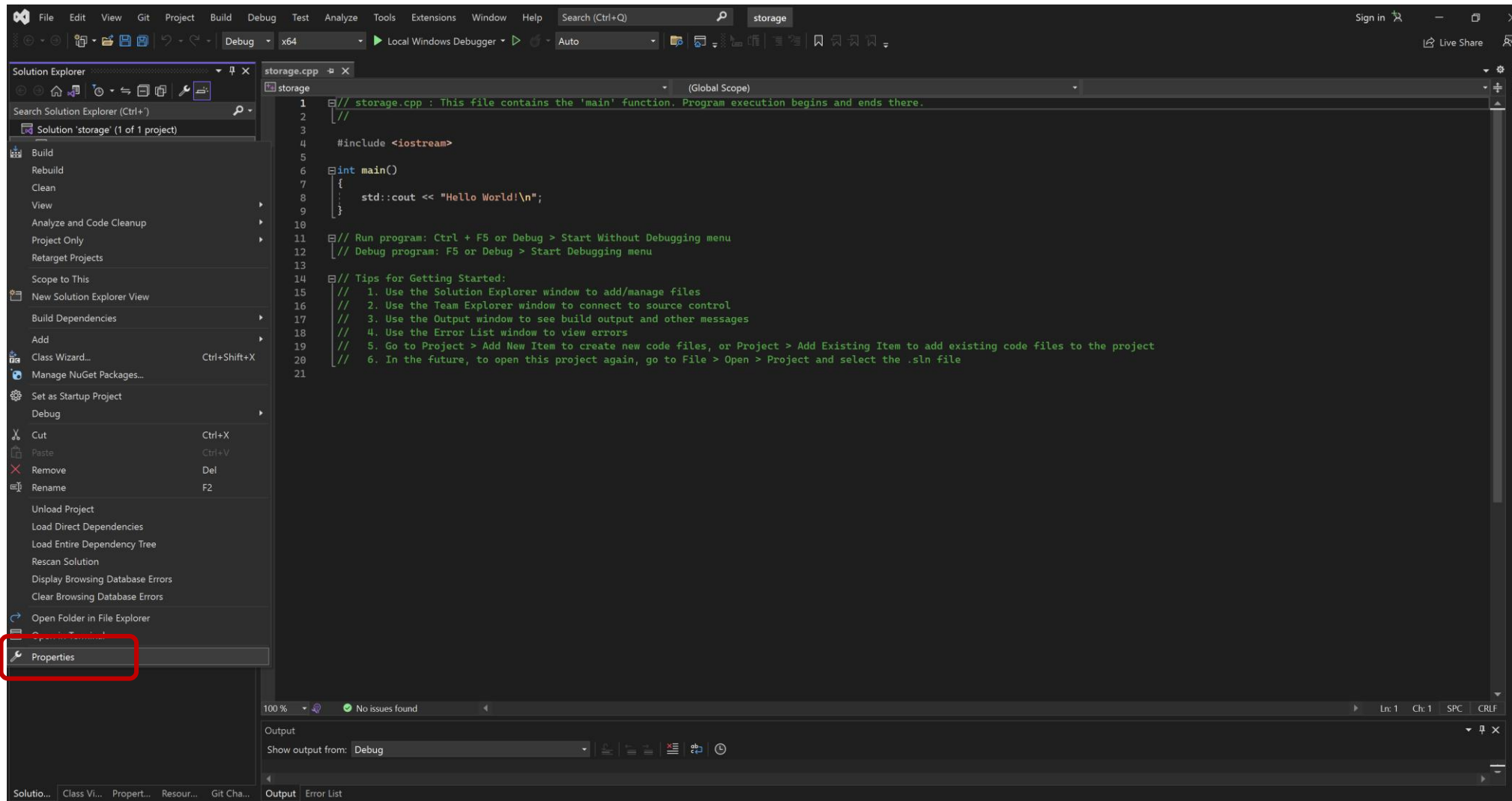
- ☐ **sim_server.c** - handles the interaction with the real kit or the simulator
- ☐ **interface.c** - allows switching between simulator and real kit.
- ☐ **mongoose.c** – embedded Web Server



CONFIGURE PROJECT PROPERTIES



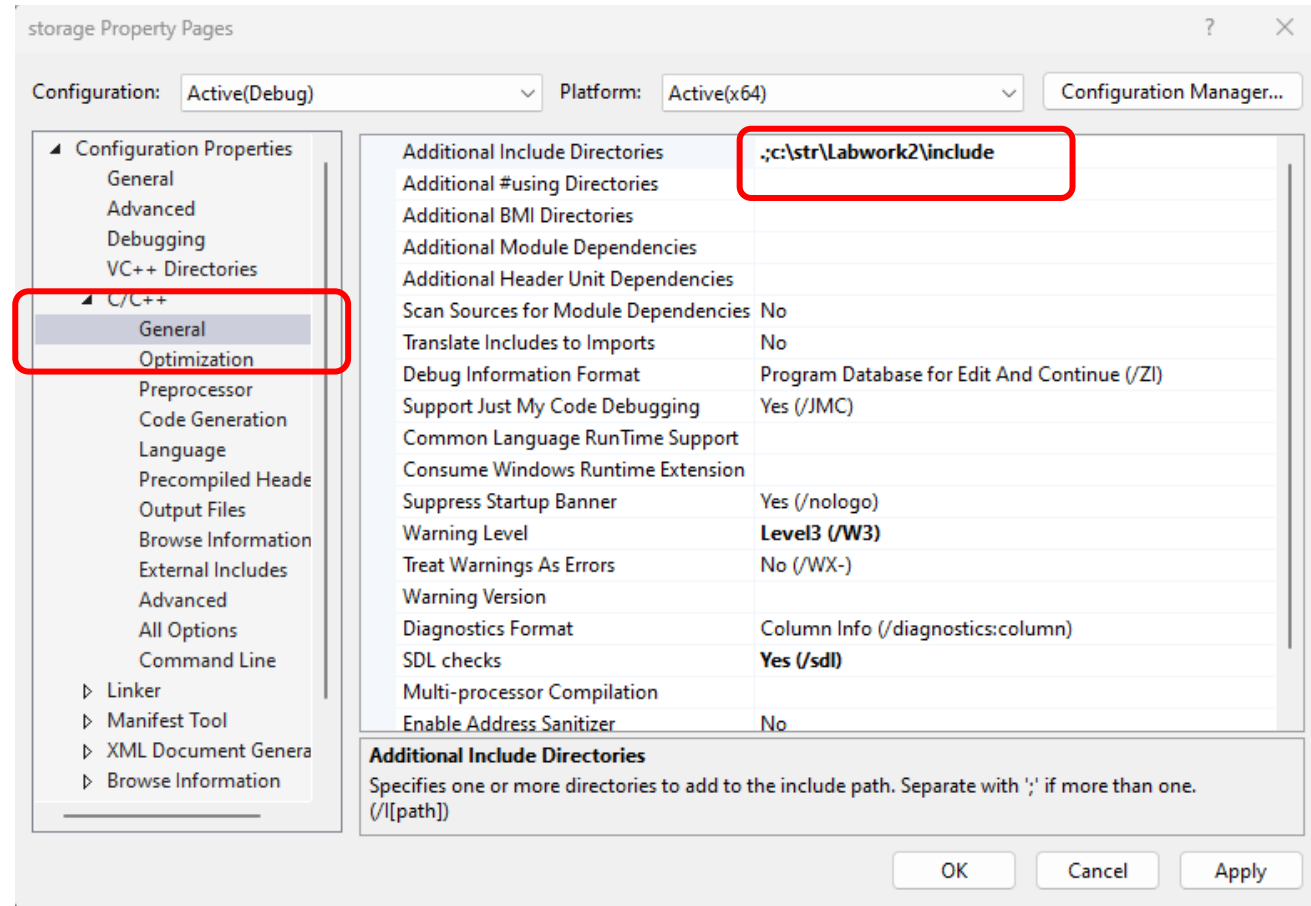
Right-click **storage** → Select **Properties**



Configure include directories:

☐ Select C/C++ -> General

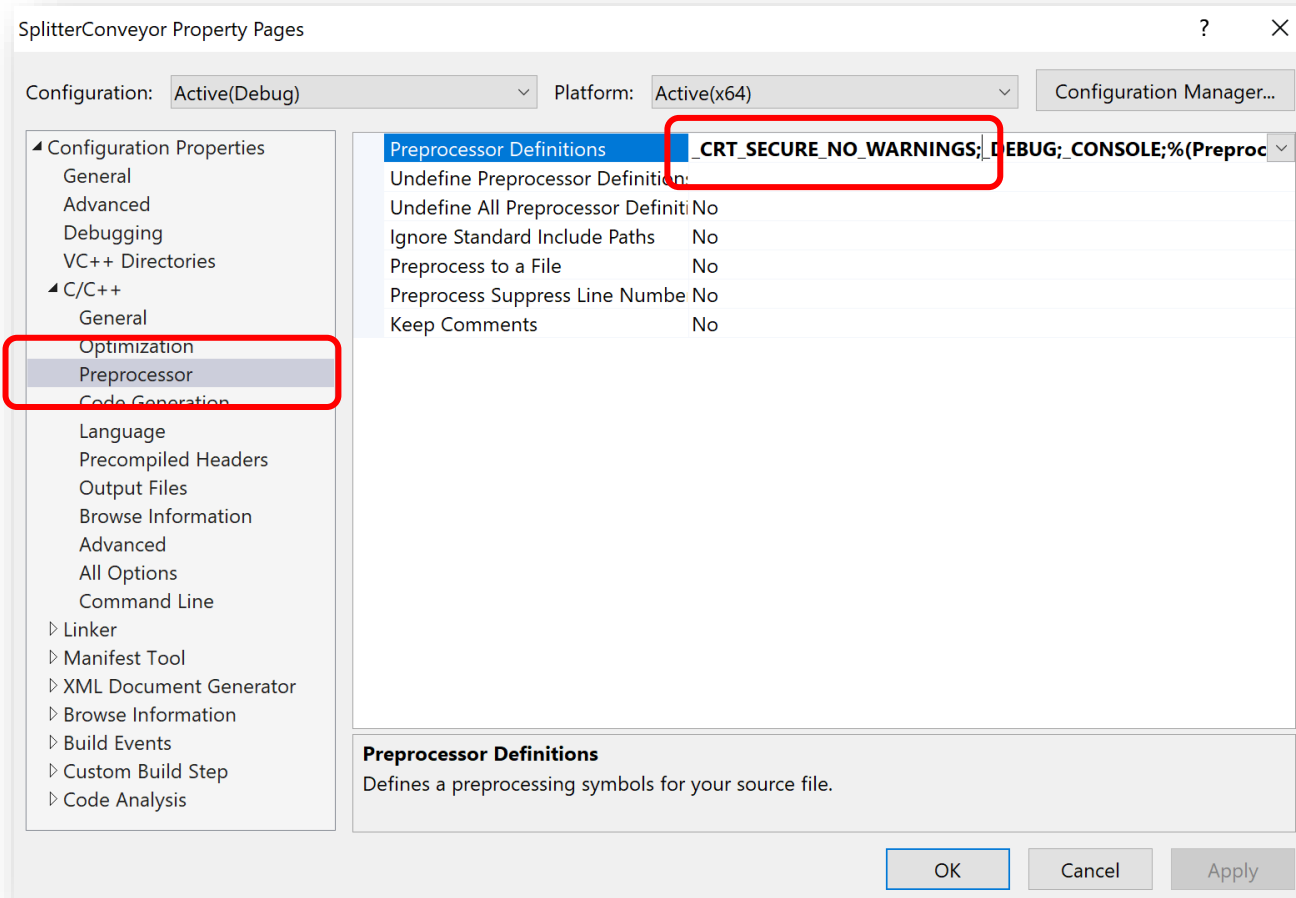
☐ Copy the include directories:
.;c:\str\Labwork2\include



CONFIGURE PROJECT PROPERTIES



Add “_CRT_SECURE_NO_WARNINGS” (in Project properties):



Copy past from here:

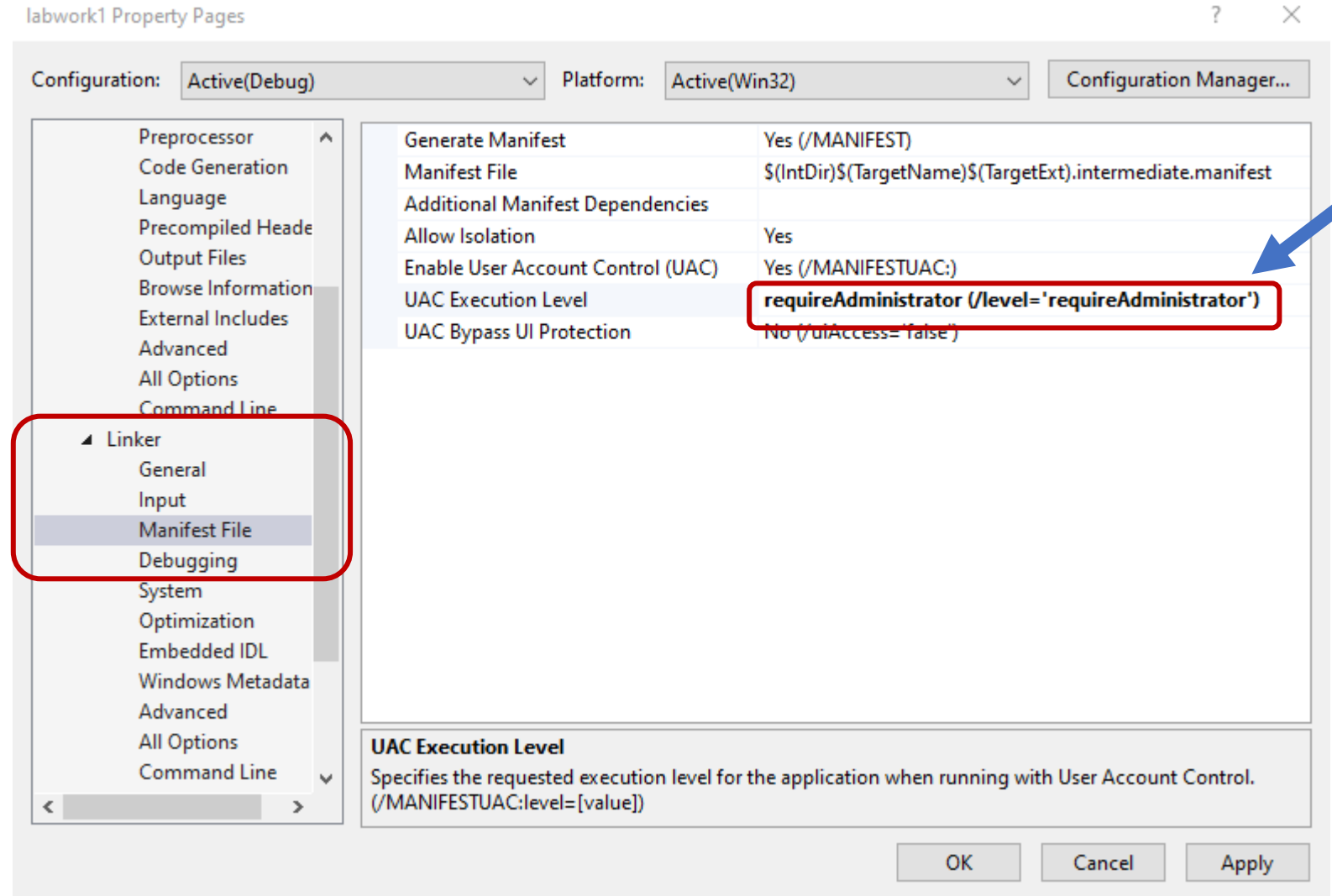
`_CRT_SECURE_NO_WARNINGS;`

CONFIGURE PROJECT PROPERTIES



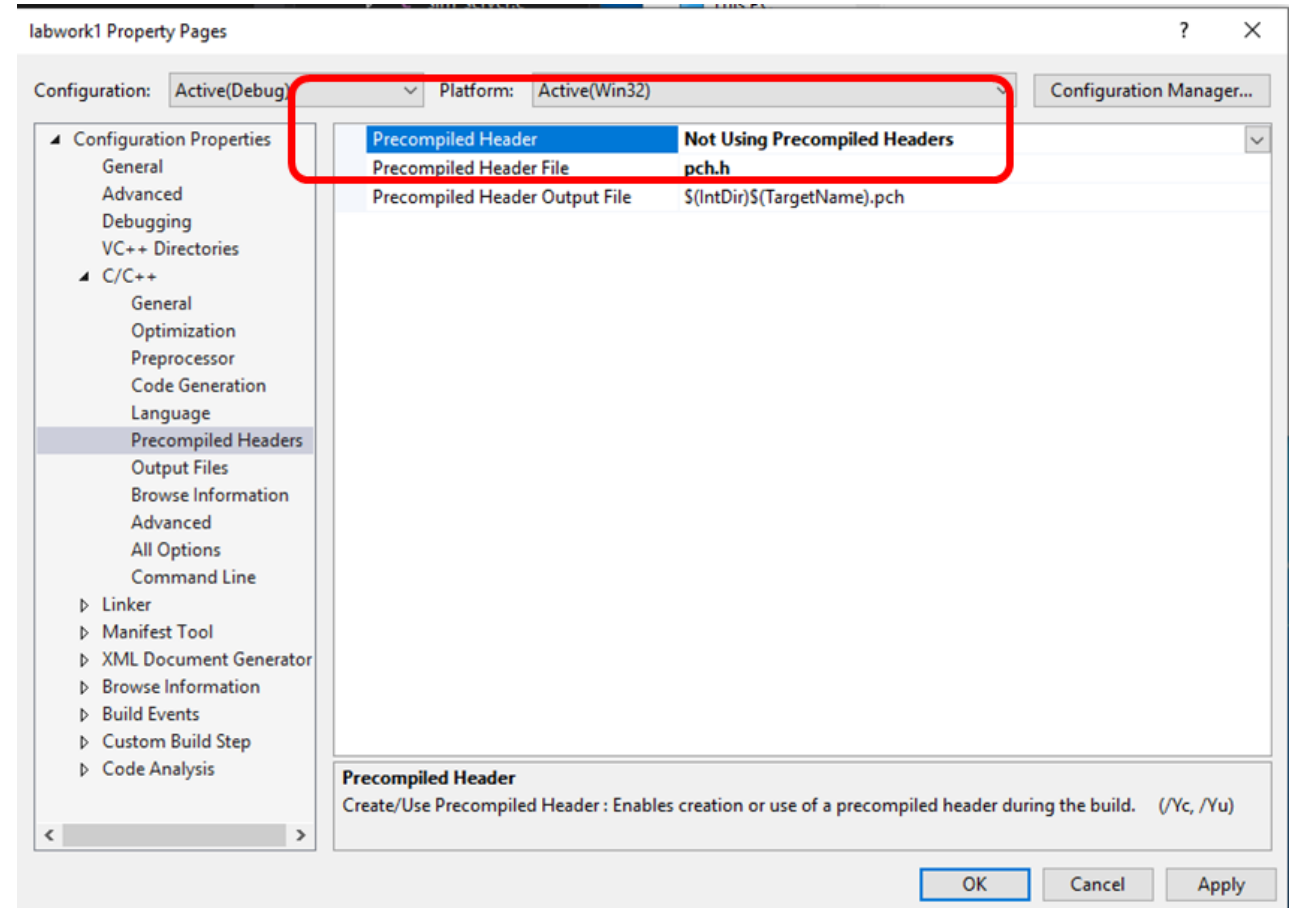
- ❑ Select Linker -> Manifest File
- ❑ Change UAC Execution level to “requireAdministrator” (*)

(*) As mentioned before, your application will receive more privileges from Windows O.S., necessary for adequate real-time behavior.



CONFIGURE PROJECT PROPERTIES

- ☐ Change pre-compiled headers option
- ☐ Select “Not Using Precompiled Headers”
- ☐ Using it would be important in big projects
(with many modules)



Try this example:

// Copy this code inside your source file
(storage.cpp)

```
#include <stdio.h>
#include <conio.h>
#include <stdio.h>

extern "C" {
    // observe your project contents. We are mixing C files with cpp ones.
    // Therefore, inside cpp files, we need to tell which functions are written in C.
    // That is why we use extern "C" directive
    #include <interface.h>
}

int main()
{
    printf("Welcome to the Compact Storage Application\n");
    printf("press key");

    createDigitalInput(0);
    createDigitalInput(1);
    createDigitalOutput(2);
    printf("\nPress a key to start");

    _getch();

    writeDigitalU8(2, 0x04);

    printf("\nPress a key to finish");

    _getch();

    closeChannels();
    return 0;
}
```

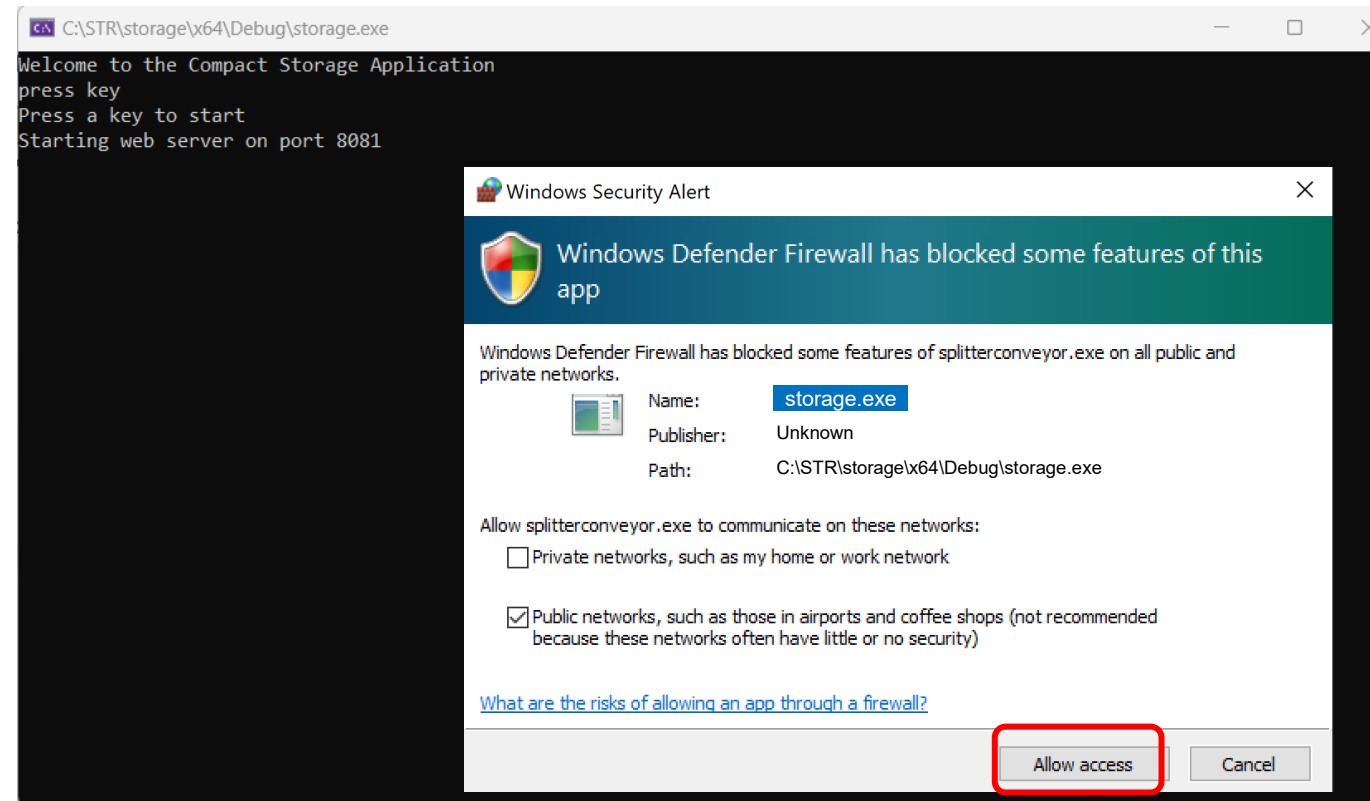

CREATION OF VS CONSOLE APPLICATION PROJECT



Trying the application:

- ☐ Run the application
- ☐ Press the allow access button
- ☐ Go into a browser and navigate to the address

http://localhost:8081/compact_storage.html

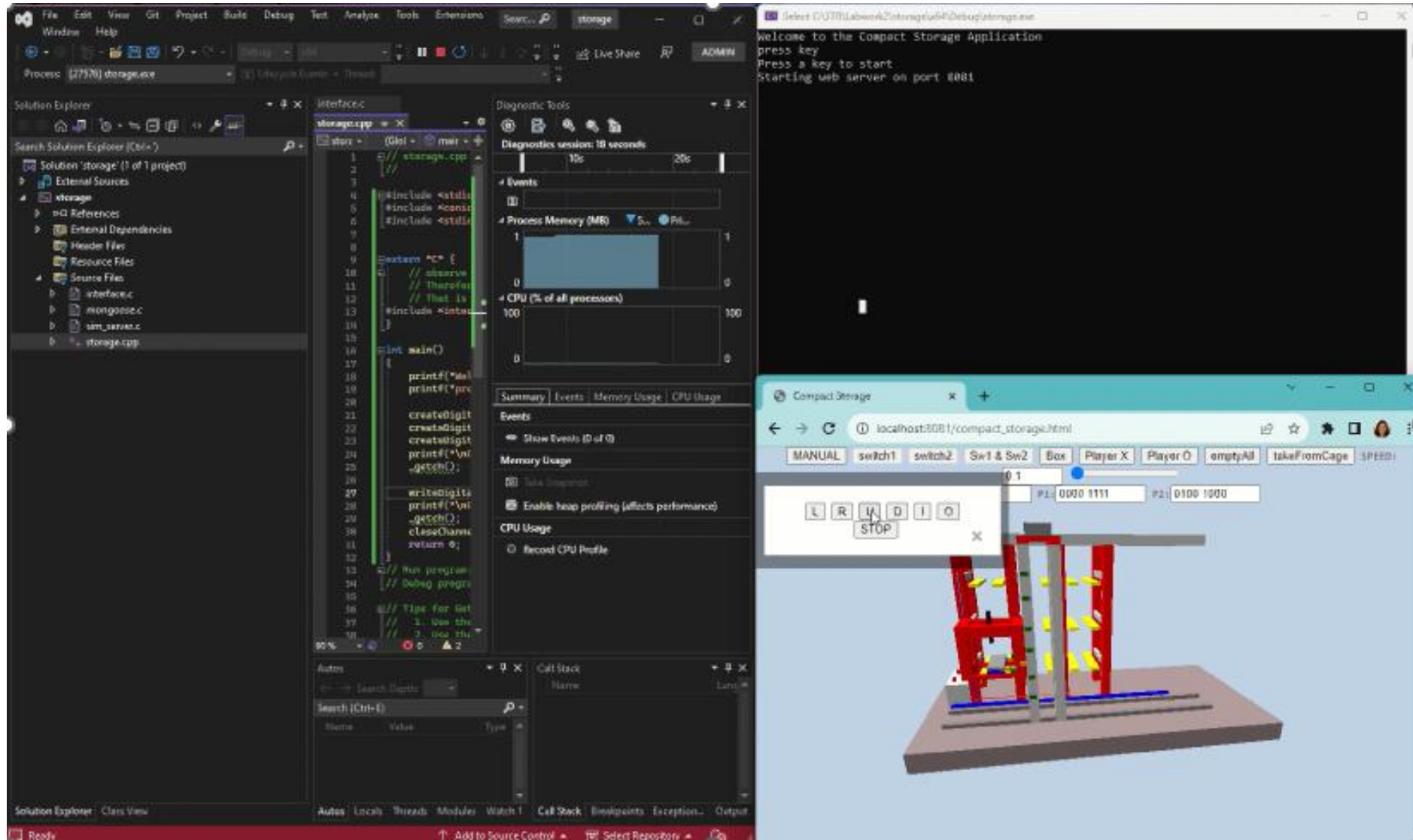


CREATION OF VS CONSOLE APPLICATION PROJECT



Trying the application:

http://localhost:8081/compact_storage.html



Case page not found:

- ❑ It means that simulator web server is working, but page **compact_storage.html** is not found.
 - Probably your libraries for **str\Labwork2** were copied into another folder, not the recommended one (**c:\str\Labwork2**)

- ❑ To solve, either
 - Copy your libraries to the recommend folder or
 - Modify the macro **MG_HTTP_FOLDER** inside **interface.c** to the value corresponding to the location of the simulator folder, e.g., "c:/str/Labwork2/simulators"

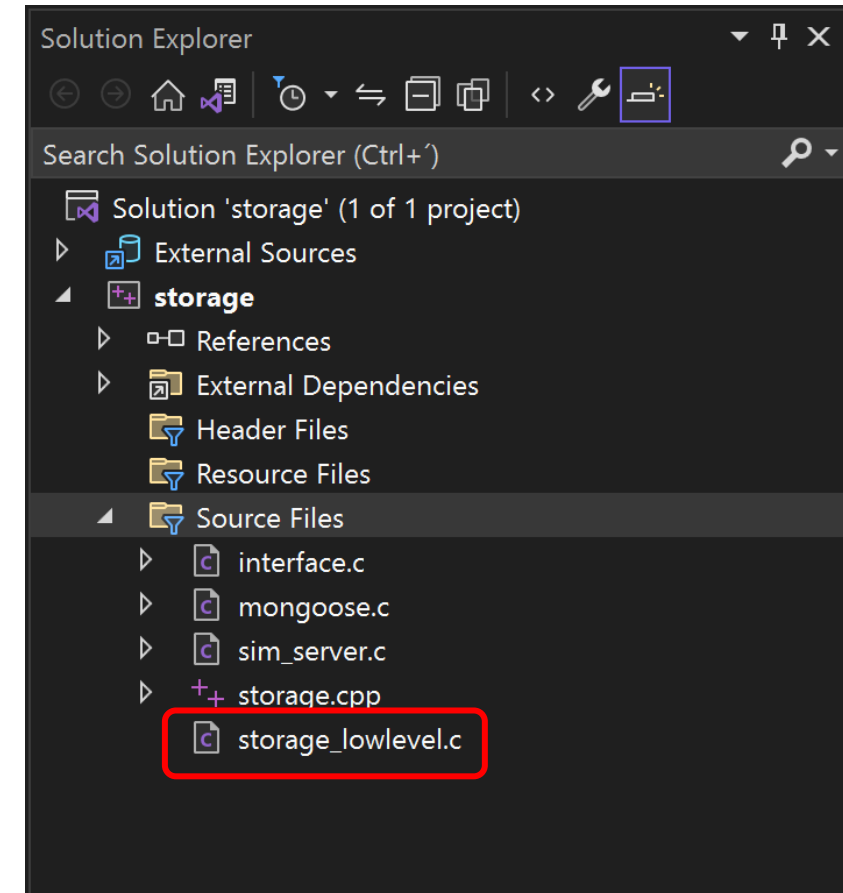
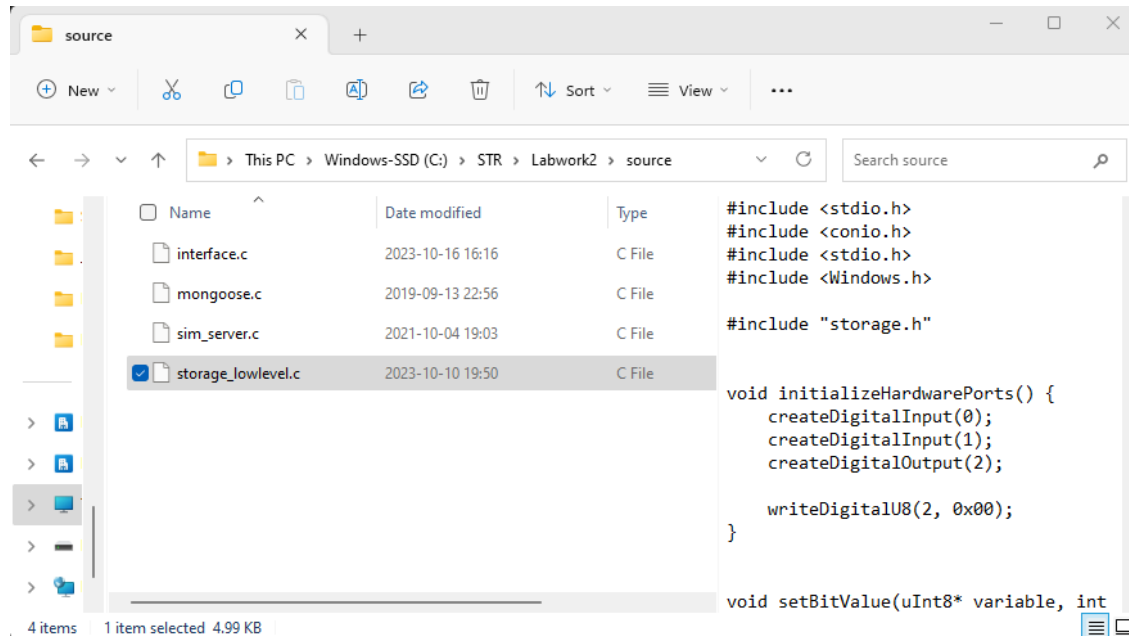


ADD SOURCE FILE TO THE PROJECT



Add source file:

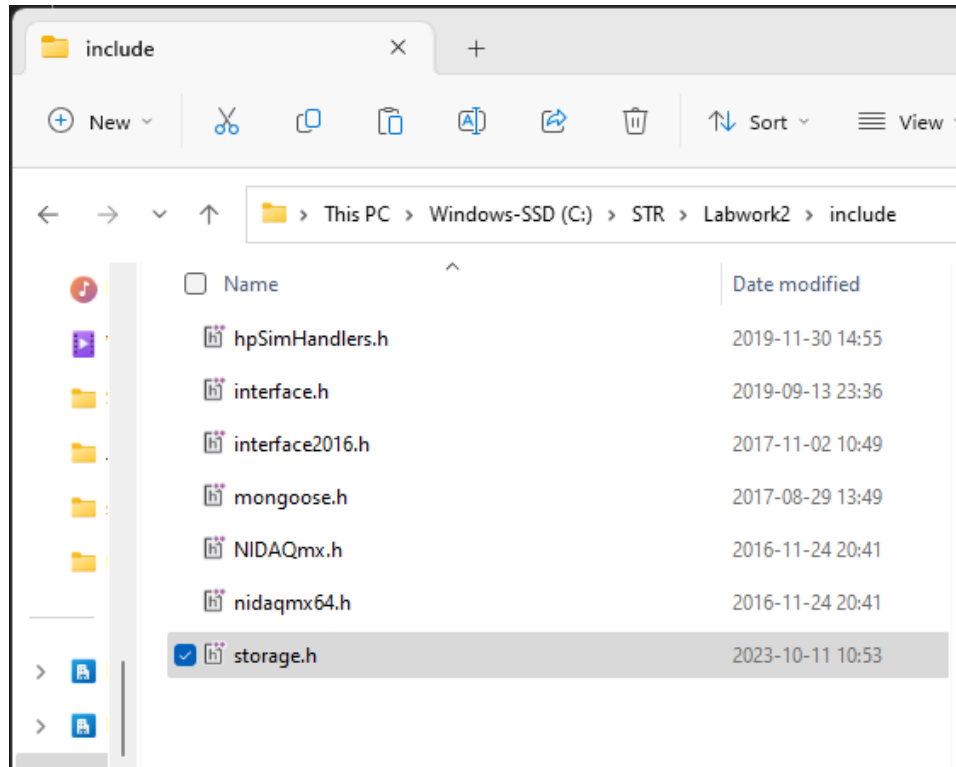
- ☐ Download from CLIP package **STR_TP2_Storage_Low_Level.zip**
- ☐ Copy file [storage_lowlevel.c](#) to source folder
- ☐ Right click the project and select to add an existing item
- ☐ Add the file [storage_lowlevel.c](#)



ADD HEADER FILE TO THE PROJECT

Add new header file to project:

- ❑ Copy header file [storage.h](#) into the include folder.



Provided low-level functions:



TEST STORAGE LOW LEVEL FUNCTIONS



```
storage (Global Scope)
1 #include <stdio.h>
2 #include <conio.h>
3
4 extern "C" {
5     // observe your project contents. We are mixing C files with cpp ones.
6     // Therefore, inside cpp files, we need to tell which functions are written in C.
7     // That is why we use extern "C" directive
8     #include <interface.h>
9     #include <storage.h>
10 }
11
12 /* IO Functions*/
13 void showMenu() { ... }
14
15 void showStates() { ... }
16
17 void executeControl(int keyboard) { ... }
18
19 int main(){
20     printf("Welcome to the Compact Storage Application\n");
21     printf("Choose a command");
22
23     initializeHardwarePorts();
24
25     int keyboard = 0;
26
27     showMenu();
28     while (keyboard != 27) {
29         if (_kbhit()) {
30             keyboard = _getch();
31             executeControl(keyboard);
32         }
33         else
34             keyboard = 0;
35     }
36
37     writeDigitalU8(2, 0x00);
38     closeChannels();
39     return 0;
40 }
```

- ❑ Complete the [storage.cpp](#) project application.
- ❑ Test all low-level functionalities.

C:\STR\Labwork2\storage\vx64\Debug\storage.exe

Welcome to the Compact Storage Application
press key
Starting web server on port 8081

X_movement.....:
q - right
a - left
z - stop

Z_movement.....:
w - up
s - down
x - stop

Y_movement.....:
e - inside
d - outside
c - stop

Led 1 on.....: 1
Led 2 on.....: p
Leds off.....: o

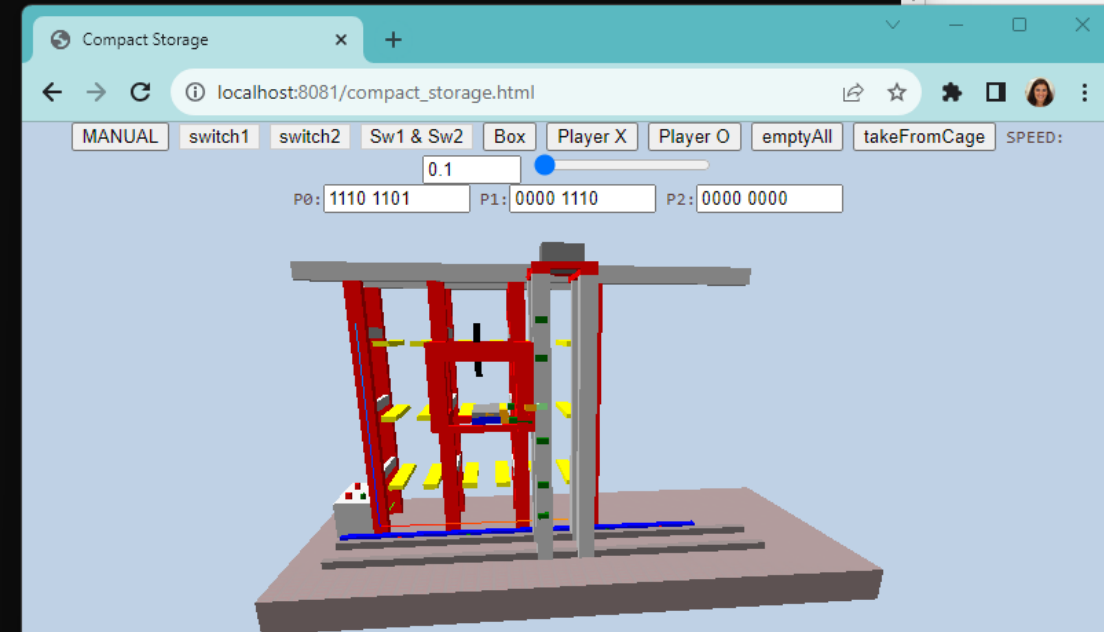
Show states.....: m

Exit.: esc

***** STORAGE STATES *****

X: 2
Z: 20
Y: 2

```
95 closeChannels();
96 return 0;
97 }
```



- ☐ JNI Development for Interacting with the Physical System
- ☐ JAVA Real-Time



KEEP UP THE
GOOD
WORK!